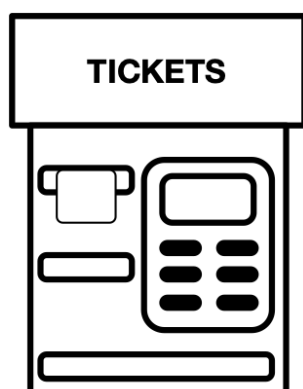




MÁQUINA DE VENDA DE BILHETES (*Ticket Machine*)

Projeto
de
Laboratório de Informática e Computadores
2025 / 2026 verão

Grupo 03:
52698 - Diogo Rodrigues;
53581 - Gonçalo Alves;
53410 - Luís Alves.
David Velez, Nuno Sebastião, Pedro Miguens, Rogério Rebelo, Rui Duarte

12 de junho de 2026

Conteúdo

1	Descrição	1
2	Arquitetura do Sistema	3
2.1	<i>Keyboard Reader</i>	3
2.1.1	<i>Key Decode</i>	4
2.1.2	Ring Buffer	9
2.1.3	Key Transmitter	15
2.1.4	Vista geral do funcionamento	18
2.2	<i>Coin Acceptor</i>	19
2.3	<i>Port Expander LCD</i>	20
2.3.1	Serial Receiver	21
2.4	<i>Port Expander Ticket Dispenser</i>	21
2.5	<i>Control</i>	22
2.5.1	Hardware Abstract Layer	23
2.5.2	KBD	23
2.5.3	Key Receiver	23
2.5.4	LCD	23
2.5.5	Ticket Dispenser	24
2.5.6	Serial Emitter	24
2.5.7	Coin Acceptor	24
2.5.8	Coin Deposit	24
2.5.9	M	24
2.5.10	Stations	24
2.5.11	File Access	24
2.5.12	Text User Interface	25
2.5.13	Ticket Machine APP	25
3	Resultados Experimentais	26
4	Conclusões	28
5	Trabalho Futuro	28
	Referências	29
A	Ticket Machine - Hardware	30
A.1	TicketMachine.vhd	30
A.2	Atribuição de Pinos	34
B	Ticket Machine - Software (<i>Control</i>)	34
B.1	Ticket Machine - App	34
B.2	M	45
B.3	TUI	46

B.4 Coin Acceptor	54
B.5 Coin Deposit	58
B.6 Stations	62
B.7 File Access	64
B.8 KBD	69
B.9 Key Receiver	71
B.10 HAL	73
B.11 Serial Emitter	76
B.12 Ticket Dispenser	78
B.13 LCD	80
B.14 Ports	85
B.15 Others	86
B.16 Custom Chars	87

Lista de Figuras

1	Diagrama de blocos da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	1
2	Arquitetura do sistema que implementa a máquina de venda de bilhetes (<i>Ticket Machine</i>)	3
3	Diagrama de blocos do <i>Keyboard Reader</i>	4
4	Diagrama de blocos do módulo <i>Key Decode</i>	4
5	Diagrama de blocos do módulo <i>Key Scan</i>	5
6	Algoritmo da máquina de estados do <i>Key Control</i>	7
	(a) Fase 1: Sem <i>repeat</i>	7
	(b) Fase 2: Com <i>repeat</i>	7
7	Diagrama de blocos do módulo <i>TDelay_Timer</i>	8
8	Diagrama de blocos do <i>Ring Buffer</i>	10
9	Diagrama de blocos do <i>Memory Address Control</i>	10
10	<i>Ring Buffer</i> em diferentes estados	12
	(a) Estado normal	12
	(b) Lógica do <i>Empty</i>	12
	(c) Lógica do <i>Full</i>	12
11	<i>Ring Buffer</i> em diferentes estados	14
	(a) Versão simplificada da ASM	14
	(b) Algoritmo da máquina de estados do <i>Ring Buffer</i>	14
12	Diagrama de blocos do <i>Key Transmitter</i>	15
13	Diagrama temporal do protocolo de transmissão	16
14	Algoritmo da Máquina de Estados do <i>Key Transmitter</i>	17
15	Diagrama de blocos do módulo <i>Keyboard Reader</i>	19
16	Bloco <i>Coin Acceptor</i>	19
	a Diagrama de blocos	19
	b Diagrama temporal	19
17	Diagrama de blocos do módulo <i>Port Expander LCD (PELCD)</i>	20
18	Diagrama de blocos do módulo <i>Port Expander Ticket Dispenser (PETD)</i>	22
19	Diagrama lógico da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	23

Lista de Tabelas

1	Codificação do valor facial das moedas	20
2	Atribuição de pinos para a FPGA [4]	91

Lista de Algoritmos

1 Descrição

Este projeto implementa uma máquina de venda de bilhetes (*Ticket Machine*), que permite a aquisição de bilhetes de comboio. O percurso é definido pela estação de origem, local de compra do bilhete e pela seleção do destino digitando o identificador da estação ou através das teclas ↑ e ↓, sendo exibido no ecrã além do identificador da estação de destino o preço e o tipo de bilhete (ida ou ida/volta). A ordem de aquisição é dada através da pressão da tecla de confirmação, sendo impressa uma unidade do bilhete exibido no ecrã. A máquina não realiza trocos e só aceita moedas de: 0,05€; 0,10€; 0,20€; 0,50€; 1,00€; e 2,00€. Para além do modo de Dispensa, o sistema tem mais um modo de funcionamento designado por Manutenção, que é ativado por uma chave de manutenção. Este modo permite o teste da máquina de venda de bilhetes, bem como permite iniciar e consultar os contadores de bilhetes e moedas. A máquina de venda de bilhetes é constituída pelo sistema de gestão (designado por *Control* na Figura 1) e pelos seguintes periféricos: um teclado de 16 teclas, um moedeiro (designado por *Coin Acceptor*), um ecrã *Liquid Cristal Display* (LCD) de duas linhas de 16 caracteres, um mecanismo de dispensa de bilhetes (designado por *Ticket Dispenser*) e uma chave de manutenção (designada por *M*) que define se a máquina de venda de bilhetes está em modo de Manutenção, conforme o diagrama de blocos apresentado na Figura 1.

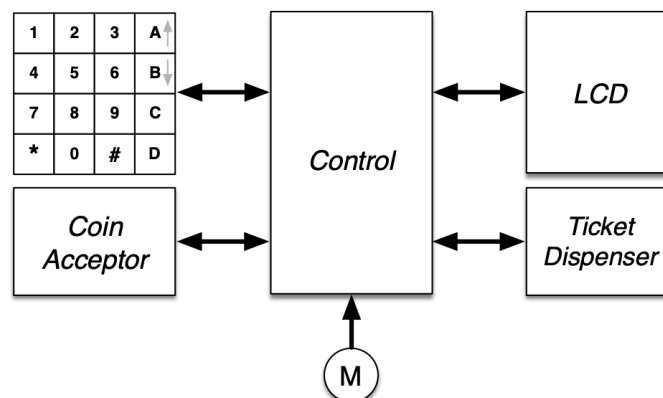


Figura 1: Diagrama de blocos da Máquina de Venda de Bilhetes (*Ticket Machine*)

Sobre o sistema podem-se realizar as seguintes ações em modo Venda:

- **Consulta e venda** A consulta de um bilhete é realizada digitando o identificador da estação de destino ou listando-a através das teclas ↑ e ↓. O processo de compra do bilhete inicia-se premindo a tecla '#'. Durante a inserção do respetivo valor monetário, é possível alterar o tipo de bilhete (ida ou ida/volta) premindo a tecla '*', com a consequente alteração do preço da viagem afixado no *LCD*, duplicando o valor no caso de ida/volta. Durante a compra ficam afixados no *LCD* as informações referentes ao bilhete pretendido até que o mecanismo de impressão de bilhetes confirme que a impressão já foi realizada e a recolha do bilhete efetuada. A compra pode ser cancelada premindo a tecla '#', devolvendo as moedas inseridas.

Sobre o sistema podem-se realizar as seguintes ações em modo Manutenção:

- **Teste** – Esta opção do menu permite realizar um procedimento de consulta e venda de um bilhete, sem introdução de moedas e sem esta operação ser contabilizada como uma aquisição.
- **Consulta Bilhetes** – Premindo a tecla 'A', em seguida através das teclas ↑ e ↓, visualizam os contadores do número de bilhetes vendidos.
- **Consulta Moedas** – Premindo a tecla 'B', em seguida através das teclas ↑ e ↓, visualizam os contadores das moedas no cofre.
- **Iniciar os contadores de moedas e bilhetes** – Premindo a tecla 'C' e em seguida a tecla '*', o sistema de gestão coloca os contadores de moedas e bilhetes a zero, iniciando um novo ciclo de contagem.
- **Desligar** – Premindo a tecla 'D' permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores de moedas e da lista de bilhetes vendidos, são armazenadas de forma persistente em dois ficheiros de texto. A informação em cada ficheiros deve estar organizada por linha, em que os campos de dados são separados por “;”, com o respetivo formato: 'COIN_VALUE;NUMBER_COINS' (valor facial; número de moedas) e 'PRICE;SOLD_TICKETS;STATION_NAME' (preço; número de bilhetes vendidos para a estação; nome da estação). Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

Nota: A inserção de informação através do teclado tem o seguinte critério: *i*) se não for premida nenhuma tecla num intervalo de cinco segundos o comando em curso é abortado; *ii*) quando o dado a introduzir é composto por mais que um dígito, são considerados apenas os últimos dígitos, a inserção realiza-se do dígito de maior peso para o de menor peso.

2 Arquitetura do Sistema

O sistema implementa uma solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Port Expander LCD (PELCD)*; *iii*) um módulo de interface com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*), designado por *Port Expander Ticket Dispenser (PETD)*; *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*.

Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será emulado utilizando quatro interruptores e três *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

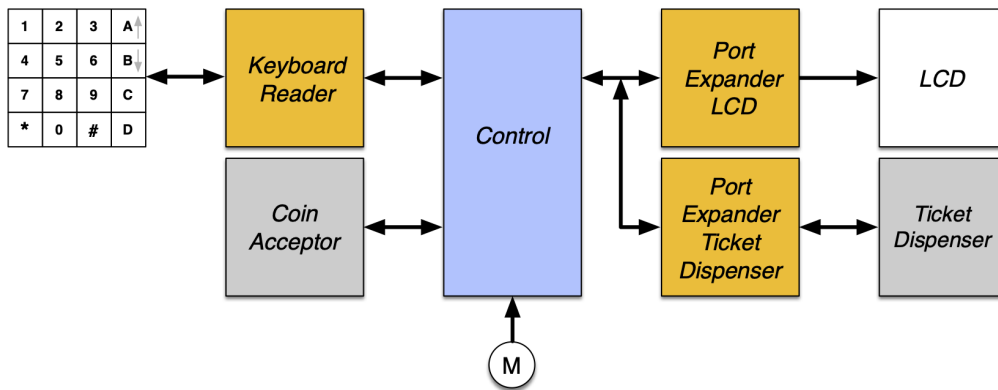


Figura 2: Arquitetura do sistema que implementa a máquina de venda de bilhetes (*Ticket Machine*)

O módulo *Keyboard Reader* é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos. O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *PELCD*. O *Ticket Dispenser* é atuado pelo módulo *Control*, através do módulo *PETD*. Por forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *PELCD* e *PETD* é realizada através de um protocolo série.

2.1 *Keyboard Reader*

O módulo *Keyboard Reader* [1] é constituído por três blocos principais: *i*) o decodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*). Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

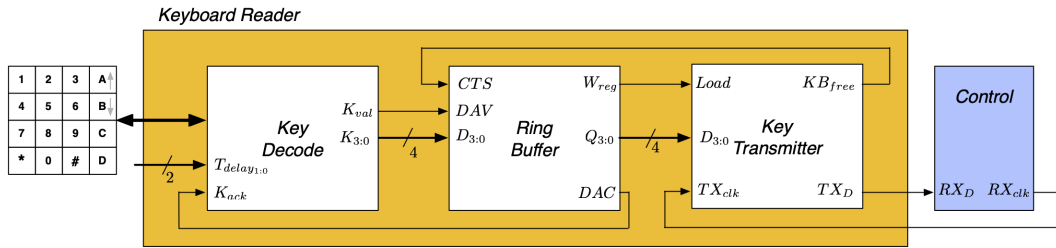


Figura 3: Diagrama de blocos do *Keyboard Reader*

2.1.1 Key Decode

O bloco *Key Decode* é responsável por detetar e processar as teclas premidas num teclado matricial 4x4 (16 teclas). O varrimento é feito pela leitura das linhas e das colunas. Esta unidade integra 3 componentes: *i)* *Key Scan*, que faz o varrimento, percorrendo e decodificando o teclado matricial; *ii)* *Key Control*, a máquina de estados que descreve o controlo de fluxo; e *iii)* *TdelayTimer*, o temporizador responsável por gerar *auto repeat*.

O módulo foi construído conforme apresentado no diagrama da Figura 4.

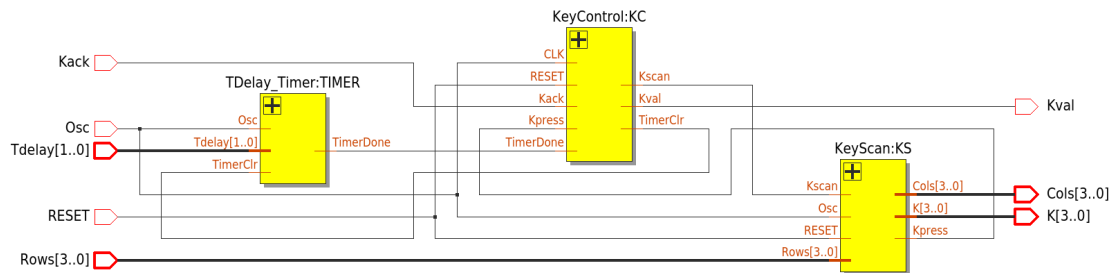


Figura 4: Diagrama de blocos do módulo *Key Decode*

i) Key Scan

O bloco *KeyScan* percorre todas as teclas do teclado, deteta a pressão de uma tecla, decodifica e retorna o seu respetivo código em 4 bits. O varrimento é realizado percorrendo as linhas na respetiva coluna, e avançando para a próxima coluna, e assim sucessivamente. Ativa ciclicamente cada coluna do teclado e verifica quais linhas estão ativas (*LOW*) para detetar qual tecla foi premida. O código da tecla é do formato *ccll*, ou seja, os dois *MSB* codificam a coluna e os dois *LSB* codificam a linha. Por exemplo, a tecla 5 corresponde a 0101 (Linha 1 e coluna 1). Este bloco integra 3 componentes para realizar as suas funções: *i)* *Counter*; *ii)* *MUX4 : 2*; *iii)* *Decoder*.

Counter

Contador de 4 bits que avança em todas as posições do teclado. Incrementa sem parar enquanto o módulo está no estado de varrimento ($Kscan = 1$) e nenhuma tecla for pressionada ($\overline{Kpress}$). Os seus 2 *MSB* controlam as colunas e os seus 2 *LSB* controlam as linhas. Por exemplo, sempre

que o contador chegar á linha 4 e não houver tecla premida (0011), o contador passa a 0100, voltando á linha inicial e ativando a próxima coluna.

MUX4:2

Este multiplexer é um seletor de linhas. Recebe as 4 linhas do teclado como inputs. Vai selecionando cada uma das linhas por coluna, á medida que o contador incrementa/varre, analisando o seu estado lógico e informando então, que houve pressão de tecla. Todas as linhas são a 1 por defeito, estando a linha ativa com valor lógico 0. O sinal de deteção de tecla (*Kpress*) terá, então, de ser negado.

Decoder

O decoder faz o trabalho inverso ao do multiplexer, agindo como o ativador de colunas. Ele recebe os 2 *MSB* do contador e irá ativar uma coluna de cada vez, caso ainda esteja a varrer.

- 00UU - Ativa a primeira coluna;
- 01UU - Ativa a segunda coluna;
- 10UU - Ativa a terceira coluna;
- 11UU - Ativa a quarta coluna.

Os pinos das linhas e das colunas são ativos em nível baixo por estarem ligados á massa por redes *Pull – Up*, sendo necessário negar as saídas deste decodificador.

O módulo foi construído conforme apresentado no diagrama da Figura 5.

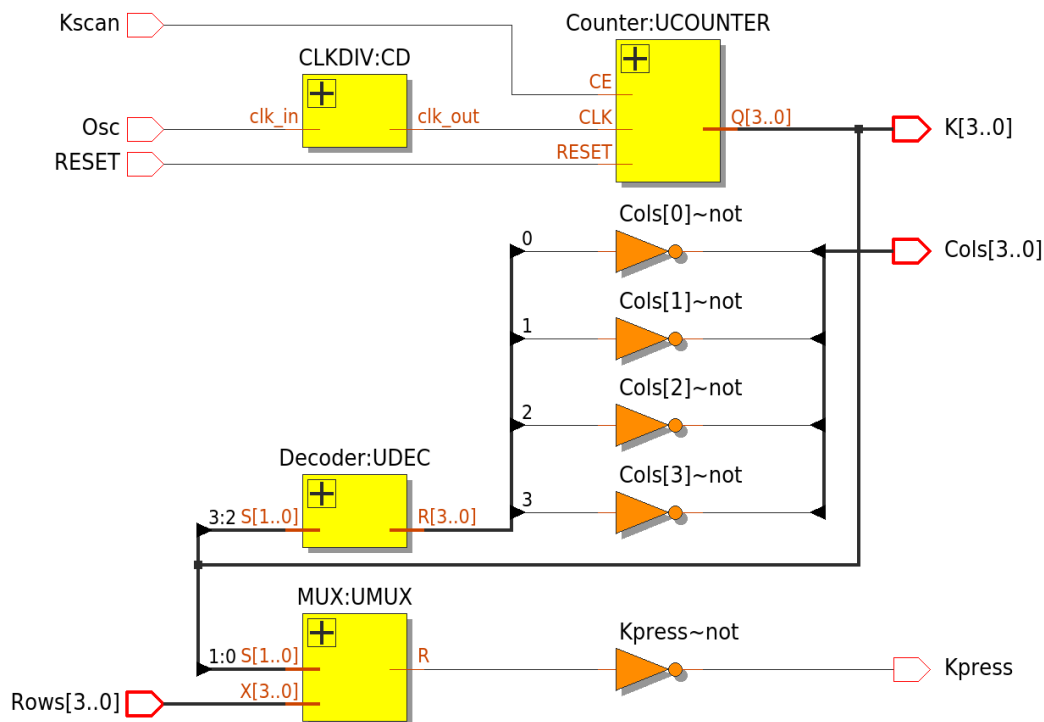


Figura 5: Diagrama de blocos do módulo *Key Scan*

Em síntese: O utilizador pressiona uma tecla, o módulo usa o contador para percorrer as teclas uma a uma, a coluna correspondente é ativa, a linha correspondente vai a 0 e retorna o sinal

Kpress, assim como o código binário da tecla, o valor de contagem.

ii) Key Control

O módulo *KeyControl* é responsável por coordenar o sistema, nomeadamente entre o *KeyDecode* e o *RingBuffer*. As suas funções consistem em gerir o ciclo de varrimento, validação e repetição de teclas. Esta ASM é implementada de forma estrutural através de dois *FlipFlops*, permitindo mapear um total de 4 estados lógicos codificados em binário (*Q1* e *Q0*): ***i) SCANNING***, o estado de repouso em o sistema está no modo de varrimento; ***ii) SENDING***, o estado de disponibilização de dados; ***iii) SENT***, o estado de retenção; e ***iv) REPEAT***, o estado de temporização para repetição automática.

Estado 00 - SCANNING

Neste estado é iniciado o ciclo de varrimento do teclado com a ativação do sinal *Kscan*. O varrimento é ativado com a ligação deste sinal ao *Count Enable (CE)* do contador, permitindo que incremente a cada flanco de súbida do relógio. A partir deste ponto, há apenas transição de estado caso o sinal *Kpress* esteja no valor lógico 1, ié, foi detetada uma tecla pressionada (indicando que o MUX detetou a linha da tecla). *Kpress* a 1 força o sistema a transitar para o estado 01 (*SENDING*).

Estado 01 - SENDING

Neste estado, com uma tecla disponível a ser enviada, é interrompido o varrimento, descendo o sinal *Kscan*, com o objetivo de preservar a tecla. Simultaneamente é ativado o sinal *Kval*, que alerta o *Ring Buffer* que há uma nova tecla a ser lida. A ASM bloqueia neste estado enquanto o recetor (*Ring Buffer*) não responder com o sinal de reconhecimento (*Kack* = 1), garantindo que nenhuma tecla é perdida. Após o reconhecimento, transita então para o estado 10 (*SENT*).

Estado 10 - SENT

Este estado é uma zona de segurança para garantir a melhor comunicação entre o *Ring Buffer* e o *Key Decode*. Só transita para o próximo estado caso o recetor volte a colocar *Kack* ao estado lógico 0. Isto garante que o recetor processou a tecla e libertou o barramento. Sem este estado, se o relógio do sistema fosse muito rápido, a máquina poderia interpretar o mesmo *Kack* como uma nova validação. Com *Kack* a 0, transita então para o estado 11 (*REPEAT*).

Estado 11 - REPEAT

Neste estado o sinal *TimerClr* é finalmente colocado no valor lógico 0, estando a 1 nos 3 estados anteriores. Com o *clear* a 0, este é o estado em que o temporizador de 0.5s (*TdelayTimer* começa a trabalhar, contando *ticks* de meio segundo até atingir o valor alvo predefinido, o *TDelay*, com o objetivo de gerar repetição automática do código da tecla, ainda premida. A transição de estado é baseada em duas condições: ***i)*** Enquanto *Kpress* é mantido a 1 (tecla não libertada), o temporizador conta até ao intervalo de repetição, e a ASM espera até o temporizador exceder o tempo, sinalizado através de *TimerDone* = 1. Quando este sinal é ativado, significa que o utilizador não libertou a tecla até então e pretende gerar uma repetição do mesmo código. Como este código precisa de ser enviado, a máquina transita então para o estado 01 (*SENDING*); ***ii)*** Se o *Kpress* descer, significa que o utilizador terá soltado a tecla. A ASM termina por aqui e regressa ao estado inicial de varrimento, á procura de novas teclas (*SCANNING*).

A figura 6 apresenta o diagram do Algoritmo da Máquina de Estados do *Key Control*.

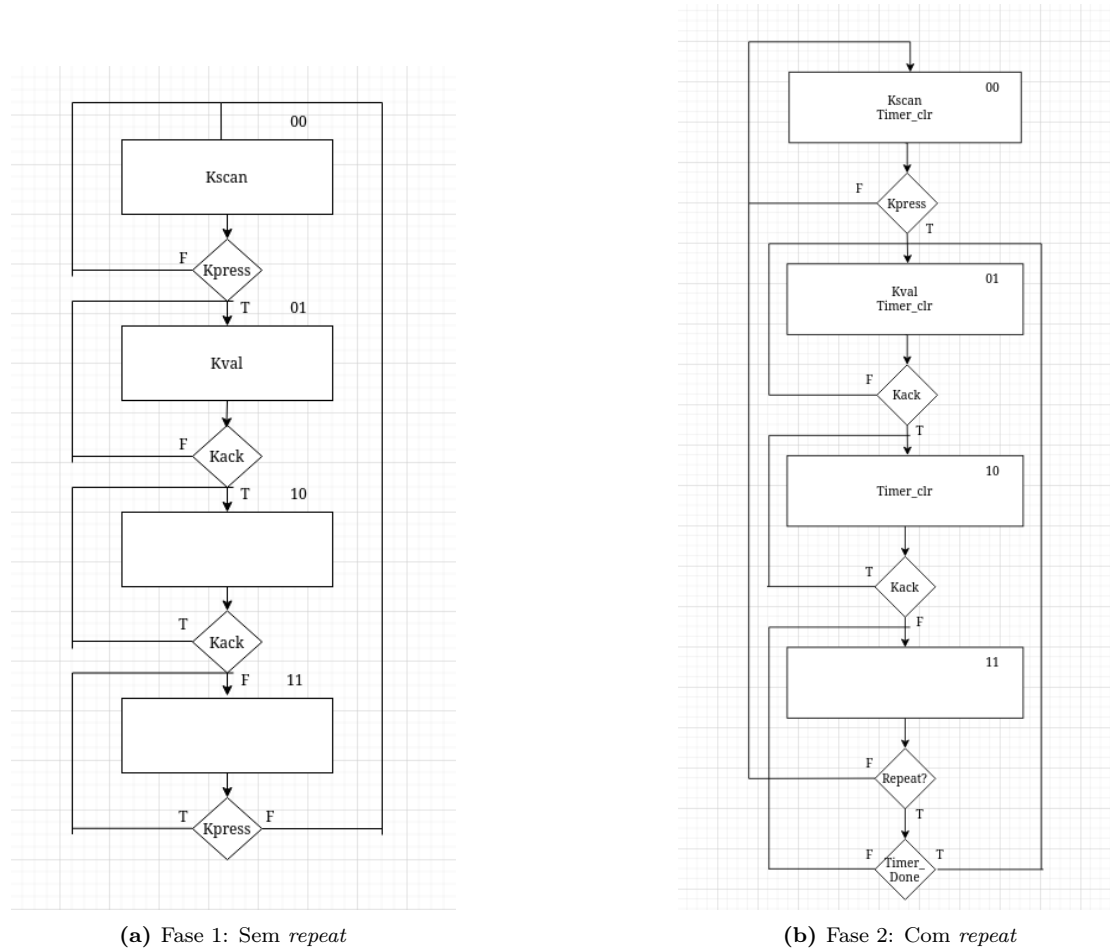


Figura 6: Algoritmo da máquina de estados do *Key Control*.

iii) *TdelayTimer*

O temporizador auxilia no mecanismo de auto repetição. A sua função consiste em dividir a frequência do *Master clock* (relógio da FPGA de 50 MHz) em 12500000, de modo a converter para períodos/*ticks* de 500ms (visto que o *Tdelay* é configurado com 500ms e os seguintes múltiplos), e contar o número de *ticks* necessários até chegar ao intervalo de repetição configurável (*Tdelay*). É importante notar que o relógio é uma onda quadrada que transita entre os níveis 1 e 0. Como os ticks ligam ao relógio do contador, que só incrementa nos flancos de subida, apenas nos interessa a distância entre dois flancos de subida. Ao dividir a frequência por 12500000, uma subida e descida, parando exatamente antes do 2^a flanco de subida, dura exatamente 500ms. Quando este período termina (*Timer_Done*=1), a ASM volta ao estado *SENDING* para gerar nova deteção da mesma tecla. Este bloco integra então dois componentes: i) *CLKDIV*, divisor de relógio configurado para 500ms; ii) *Counter*, que conta os períodos. Adicionalmente, é também implementada a lógica de comparação do valor de contagem atual (saída do *Counter*), com o valor alvo (*Tdelay*), para gerar *Timer_Done*.

CLKDIV

Divisor de frequência. Recebe como entrada o relógio de 50 MHz e divide a sua frequência pelo valor definido para gerar *ticks* desejados. Este bloco foi fornecido pelos docentes, tendo sido feitas apenas algumas adaptações para gerar *ticks* de 500ms.

Counter

Contador padrão, visto que já recebe os períodos convertidos. Logo cada incremento corresponde a 500ms. O sinal *Timer_clr* é ligado ao seu *RESET* de modo a reiniciar a contagem.

Lógica adicional

Resta verificar se o número de *ticks*/valor de contagem atingiu o valor alvo *Tdelay*:

- Se *Tdelay* = 00 (500ms) e contagem = 0001 (1 *tick* (500ms), então o tempo alvo foi atingido (*Timer_Done* = 1), caso contrário, permanece a 0;
- Se *Tdelay* = 01 (1000ms) e contagem = 0010 (2 *tick* (1000ms), então o tempo alvo foi atingido (*Timer_Done* = 1), caso contrário, permanece a 0;
- Se *Tdelay* = 10 (1500ms) e contagem = 0011 (3 *tick* (1500ms), então o tempo alvo foi atingido (*Timer_Done* = 1), caso contrário, permanece a 0;
- Se *Tdelay* = 11 (2000ms) e contagem = 0100 (4 *tick* (2000ms), então o tempo alvo foi atingido (*Timer_Done* = 1), caso contrário, permanece a 0;

O módulo foi construído conforme apresentado no diagrama da Figura 7.

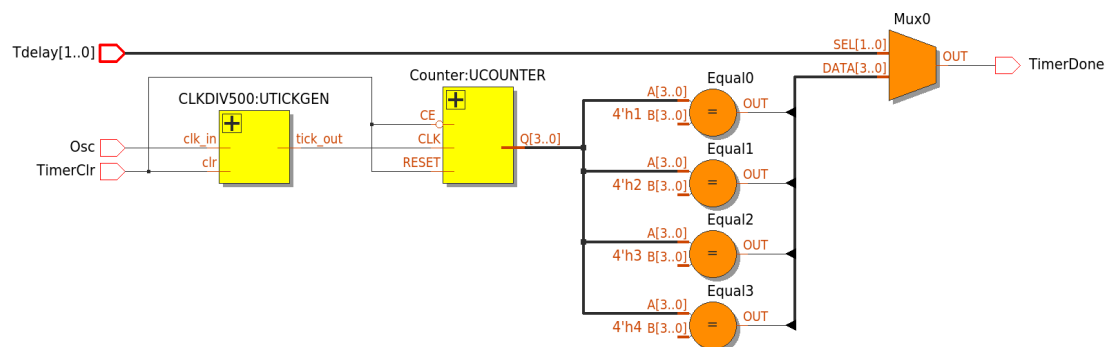


Figura 7: Diagrama de blocos do módulo *TDelay_Timer*

iv) Key Decode - Vista geral

O *Key Decode* é o módulo responsável por detetar e processar teclas num teclado matricial 4×4 (16 teclas). No seu estado de repouso está sempre em varrimento á procura de informação (*Kscan*). Quando o utilizador pressiona uma tecla, o varrimento percorre 4 linhas por coluna, ativando a coluna atual. O valor nesse contador é retornado como código de tecla, 2 bits que codificam as colunas e 2 bits que codificam as linhas. O módulo interrompe o varrimento e informa o Ring Buffer que há dados para ler (*Kval*), necessitando de uma resposta de confirmação por parte do mesmo (*Kack*) para poder avançar. O mesmo sinal precisa de voltar a 0 ou o *Key Decode* assume que o mesmo *Kack* é um novo reconhecimento. Por fim, o módulo irá ativar o temporizador interno para gerar repetições de tecla, caso o utilizador o pretenda, com o sinal

Kpress. A libertação da tecla coloca o módulo de volta ao varrimento, á espera por novos dados.

2.1.2 Ring Buffer

O *Ring Buffer* é uma estrutura de dados que implementa uma fila FIFO (*First-In, First-Out*) com capacidade de armazenar 16 palavras e de funcionamento circular. Permite que dois subsistemas com velocidades diferentes possam trocar dados de forma sincronizada, sem perda de informação (por exemplo, se o *Key Transmitter* estiver ocupado transmitindo uma tecla e o *Key Decode* detectar outra, essa deteção seria perdida sem o *Ring Buffer*). O módulo faz a gestão de 2 ponteiros na memória: *i)* O ponteiro de leitura, que aponta para o endereço de memória de onde será extraído o próximo dado a ser processado ou enviado; *ii)* O ponteiro de escrita, que aponta para o endereço de memória onde o próximo dado recebido será armazenado.

No pedido de escrita: Este módulo recebe o código do *Key Decode* a armazenar, ativando *Kval/DAV* (Disponibilização de dados). O *Ring Buffer* verifica se está cheio: se não estiver, envia o sinal de confirmação (*Kack/DAC*) de que os dados foram aceites. Com esta confirmação, o *Key Decode* baixa então o *DAV*. Ao observar *DAV* a 0, o *Ring Buffer* baixa então o sinal *DAC* para libertar. Assim, uma tecla foi armazenada na memória.

No pedido de leitura: O módulo *Key Transmitter* informa que está pronto para receber e transmitir informação (*CTS*). O *Ring Buffer* verifica então se tem informação na memória a ser lida. Caso tenha, envia o código da tecla para o *Key Transmitter* e guarda no seu registo interno com o sinal *Wreg*. Com dados a transmitir, *Key Transmitter* informa então que está ocupado, baixando o *KBfree/CTS*, baixando também o *Ring Buffer* o sinal *Load/Wreg*. O transmissor volta a ativar o sinal após o fim da transmissão.

O módulo *Ring Buffer* integra 3 componentes: *i)* *Memory Address Control*, responsável pela gestão de endereços; *ii)* *Ring Buffer Control*, a máquina de estados que faz a sincronização entre o *Key Decode* e o *Key Transmitter*; *iii)* *RAM*, a memória FIFO interna do *Ring Buffer*, capaz de guardar 16 palavras.

O módulo foi construído conforme apresentado no diagrama da Figura 8.

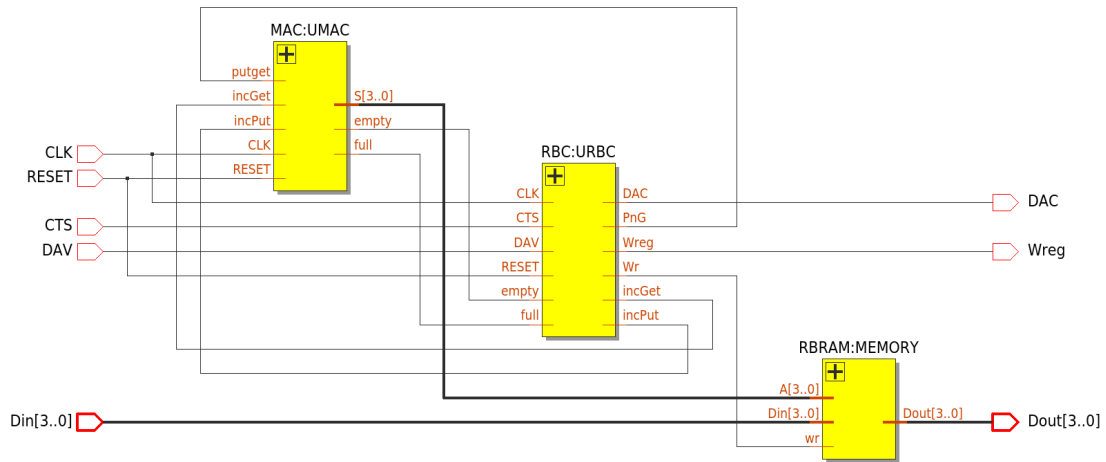


Figura 8: Diagrama de blocos do *Ring Buffer*

i) Memory Address Control

Este módulo faz a gestão de 2 ponteiros, cada um apontando para um endereço na memória: *i)* O ponteiro de leitura, que aponta para o endereço a ser lido; e *ii)* O ponteiro de escrita, que aponta para o endereço a armazenar. Este bloco avança os ponteiros, seleciona os endereços (leitura ou escrita), e adicionalmente verifica se o *Buffer* está cheio (*Full*) ou vazio (*Empty*). A memória do *Ring Buffer* é uma espécie de anel dividida em segmentos (endereços), na qual apontam, um de cada vez, os ponteiros. Assim, os ponteiros percorrem a memória de forma circular. Este módulo integra: *i)* 2 contadores de 4 bits, representando os 2 ponteiros, controlado por dois enables, *incPut* e *incGet*; *ii)* a lógica adicional de *Full* e *Empty*.

O módulo foi construído conforme apresentado no diagrama da Figura 9.

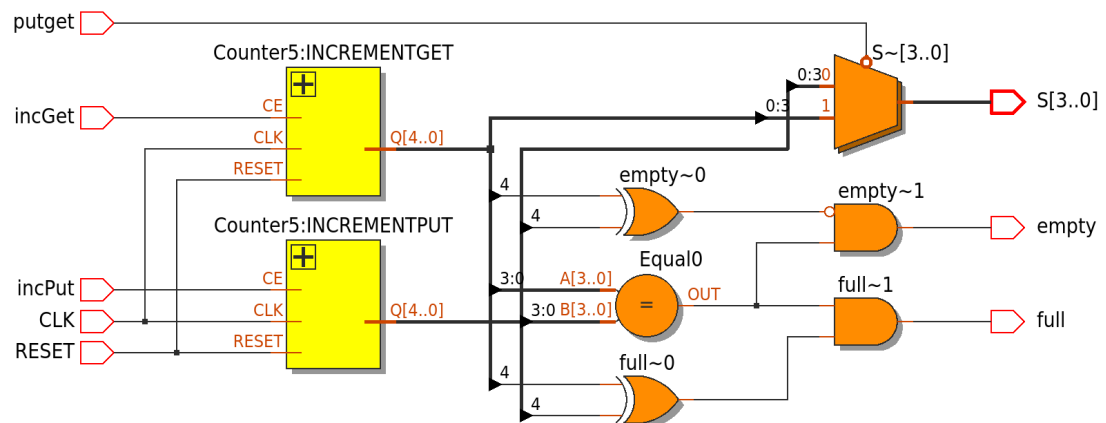


Figura 9: Diagrama de blocos do *Memory Address Control*

INCREMENTGET e INCREMENTPUT

Contadores (também funcionando como registos) padrão de 5 bits. *INCREMENTGET* refere-se ao ponteiro de leitura (*getIndex*). É ativado quando existe um pedido de leitura, ativando o módulo *Ring Buffer Control* o sinal *enable incGet*. *INCREMENTPUT* refere-se ao ponteiro de escrita (*putIndex*). É ativado quando existe um pedido de escrita, ativando o módulo *Ring Buffer Control* o sinal *enable incPut*. A cada incremento, o respetivo índice avança uma posição na memória.

Lógica Full/Empty

Por fim este módulo precisa verificar se o *Ring Buffer* está apto para responder aos pedidos de escrita e leitura, ié, verificar se está cheio (*Full*) e se pode armazenar uma palavra, ou se está vazio (*Empty*) e se há informação para ler. Os ponteiros são responsáveis por definir esta lógica, visto que são eles que indicam o estado atual da memória, onde se vai ler, onde se vai armazenar. Visto que se trata de uma memória circular, os ponteiros chegam ao último endereço (1111) e voltam ao endereço inicial (0000 - Um contador de 4 bits avança para este valor naturalmente por *Overflow*). Comparando ambos os índices, é possível saber quando o *Ring Buffer* está vazio. A memória está vazia quando o ponteiro de leitura consegue alcançar o ponteiro de escrita, e não avança desde então. Isto significa que ambos os ponteiros apontam para a mesma posição. Em partida, significa que o ponteiro de leitura, incapaz de avançar, não encontra mais dados para ler. Isto, porém, é insuficiente, pois um dos ponteiros pode já ter dado uma volta completa à memória, e caírem ambos no mesmo endereço. Por exemplo, se o ponteiro de escrita percorreu toda a memória e ambos os ponteiros apontam para o mesmo endereço, é impossível o *Buffer* estar vazio, visto que a volta feita pelo ponteiro de escrita deu a indicação para se armazenar teclas em cada posição. A solução adotada é simples: expandir o contador de 4 bits para 5 bits. Assim, o MSB dos índices sinaliza o número de voltas à memória:

- Se $putIndex[3..0] = getIndex[3..0] \cdot putIndex[4] = getIndex[4]$, a memória está vazia (*Empty*). Como estão ambos na mesma posição de memória e deram exatamente o mesmo número de voltas ao anel, não há dados novos por ler.

- Se $putIndex[3..0] = getIndex[3..0] \cdot putIndex[4] \neq getIndex[4]$, O ponteiro de escrita foi tão rápido que deu uma volta completa à memória e alcançou o ponteiro de leitura (*Get*). Eles apontam para o mesmo endereço, no entanto, o facto de o MSB ser diferente indica que o *putIndex* está exatamente uma volta à frente do *getIndex*, indicando que a memória foi preenchida.

Com um contador de 5 bits é possível então contar os endereços, e sempre que chega ao último (1111), o contador passa de 01111 para 10000, ou seja, voltou ao endereço inicial, registando também a volta.

A figura 10 descreve a lógica dos sinais.

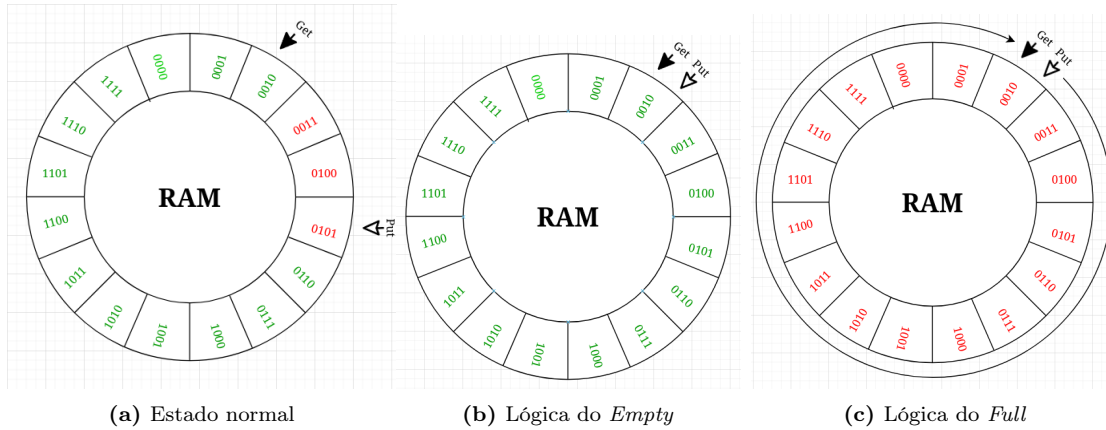


Figura 10: *Ring Buffer* em diferentes estados

ii) *Ring Buffer Control*

Este módulo é possivelmente o "coração" do *Keyboard Reader*. É a unidade de controlo encarregue de gerir os ponteiros, decidindo se o *Ring Buffer* deve ler da memória e enviar para o transmissor, ou armazenar um novo dado recebido do *Key Decode*. Esta ASM é implementada de forma estrutural através de três FlipFlops, permitindo mapear um total de 7 estados lógicos codificados em binário ($Q2$, $Q1$ e $Q0$): *i) IDLE*, o estado de repouso e monitorização dos pedidos; *ii) REQUESTPUT*, em que a máquina responde ao pedido de escrita e prepara a seleção do endereço de escrita; *iii) WRITEMEMORY*, é ativado o sinal para escrever o dado na RAM; *iv) INCREMENTPUT*, o dado foi escrito na posição corrente e o ponteiro avança para o próximo endereço; *v) ACKWRITE*, estado de confirmação de que os dados foram aceites; *vi) REQUESTGET*, o ramo de leitura: prepara a seleção do endereço de leitura; *vii) INCREMENTGET*, o dado foi lido da memória na posição corrente e o ponteiro de leitura avança para o próximo endereço.

Estado 000 - *IDLE*

Neste estado a máquina encontra-se em repouso ativo, lendo as entradas a cada flanco de relógio para verificar se há pedidos de processamento. A partir deste ponto a ASM divide-se em dois ramos, um ramo tratará do pedido de leitura e outro ramo do pedido de escrita. Um pedido de escrita é representado pelo sinal $W = DAV \cdot \overline{Full}$, ié, só é validado se existir informação disponível (DAV), e se existe forma de o *Ring Buffer* o armazenar. Um pedido de leitura é representado pelo sinal $R = CTS \cdot \overline{Empty}$, ié, só é validado se o transmissor estiver livre e existir informação na posição corrente na memória. Sendo assim, a ausência de pedidos bloqueia a ASM neste mesmo estado. Se houver um pedido de escrita (W), a ASM transita para o estado 001 (*REQUESTPUT*). Se houver um pedido de leitura (R), a ASM transita para o estado 101 (*REQUESTGET*).

Estado 001 - *REQUESTPUT*

A ASM entra no ramo de escrita. Este estado serve como um delay de segurança para assegurar que o endereço da RAM é o correto antes de iniciar a escrita. É selecionado o endereço de escrita ($PnG = 1$), e Wr é por enquanto mantido a 0. Transita incondicionalmente para o estado 010 (*WRITEMEMORY*).

Estado 010 - WRITEMEMORY

Estado em que a escrita ocorre efetivamente. A palavra vinda do *Key Decode* é guardado no segmento selecionado por PnG . Mantém PnG ativo e sobe o sinal Wr para ativar a escrita na RAM. Transita incondicionalmente para o estado 011 (INCREMENTPUT).

Estado 011 - INCREMENTPUT

Neste estado o dado já está armazenado. Uma vez que o endereço está ocupado e a memória segue a disciplina FIFO, a próxima escrita requer que o ponteiro passe sempre para o endereço seguinte. Mantém ativo o sinal $P\overline{G}$ e desce então o sinal de escrita ($\overline{Wr}$) e ativa o sinal enable responsável por incrementar o ponteiro ($incPut$). Transita incondicionalmente para o estado 100 (ACKWRITE).

Estado 100 - ACKWRITE

Este estado fecha a comunicação com o *Key Decode*, informando que recebeu a palavra e já a guardou na RAM. Faz isso ativando DAC de modo a dar *Acknowledged*/reconhecimento ao *Key Decode* ($Kack$). Este estado impede que a rapidez do relógio da FPGA voltasse ao estado IDLE com $DAV = 1$. Isso resultaria na máquina interpretar o mesmo clique como uma nova tecla e reescreveria o mesmo carácter na RAM. Este bloqueio assegura que cada pressão de tecla gera apenas uma escrita. A ASM fica então retida neste estado enquanto DAV for 1. Só transita para IDLE quando o *Key Decode* baixa o sinal DAV .

Estado 101 - REQUESTGET

Partindo do estado 000 (IDLE), temos também o ramo de leitura. Este estado é idêntico ao estado 001 (REQUESTPUT), com a diferença que o preparo é para a leitura. Coloca PnG a nível baixo de modo que o seletor aponte para o ponteiro *Get*. Ativa também o sinal $Wreg$ para que o transmissor possa ler e armazenar o código. Transita incondicionalmente para o estado 110 (INCREMENTGET).

Estado 110 - INCREMENTGET

Neste ponto o transmissor já guardou o código no seu registo interno. Com esta posição lida e liberada, o ponteiro avança então para o próximo endereço. Para isso ativa o sinal $incGET$. Transita incondicionalmente para o estado 000 (IDLE).

A figura 11 apresenta o diagrama do Algoritmo da Máquina de Estados do *Ring Buffer Control*.

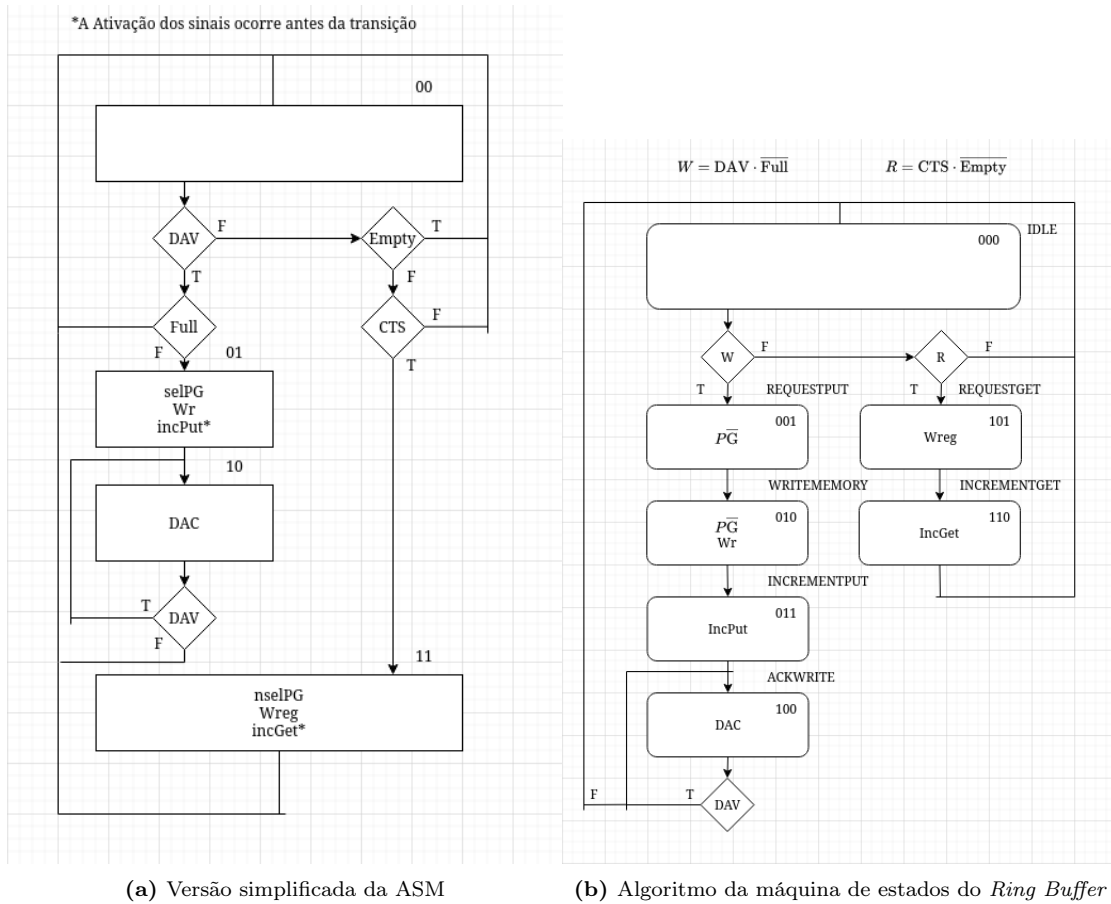


Figura 11: *Ring Buffer* em diferentes estados

iii) *Ring Biffer* - Vista Geral

O *Ring Buffer* é o módulo responsável pelo armazenamento temporário e ordenado dos códigos de tecla gerados, segundo a disciplina FIFO (*First-In, First-Out*), garantindo a sincronização entre o *Key Decode* (produtor) e o sistema central (consumidor). No seu estado de repouso, o módulo permanece à espera de que novos dados fiquem disponíveis ou de um pedido de leitura por parte do consumidor. Quando o *Key Decode* sinaliza a presença de uma nova tecla ativa através do sinal *Kval*, o *Ring Buffer* verifica o estado de ocupação da sua memória RAM interna. Caso o buffer não esteja cheio, o código de 4 bits é escrito na posição indicada pelo ponteiro de escrita, o qual é posteriormente incrementado. Ato contínuo, o módulo responde ativando o sinal de confirmação *Kack*; o *Ring Buffer* mantém este sinal ativo até que *Kval* retorne a 0, prevenindo assim a escrita duplicada do mesmo dado. Em simultâneo, sempre que existirem elementos guardados na memória (ou seja, quando os ponteiros de escrita e leitura diferem), o módulo ativa o sinal *DAV* (*Data Available*) para notificar o consumidor. A leitura da informação é coordenada pelo sinal *CTS* (*Clear to Send*) proveniente do exterior: ao ser ativado, o dado presente no endereço do ponteiro de leitura é disponibilizado no barramento de saída, e o respetivo ponteiro é incrementado. O esvaziamento completo da memória desativa o sinal *DAV*, colocando o *Ring Buffer* novamente em estado de repouso, pronto para novos ciclos de escrita ou leitura.

2.1.3 Key Transmitter

O *Key Transmitter* é o módulo responsável por transmitir, de forma série, os códigos das teclas armazenados no *Ring Buffer* para o software de controlo. A transmissão é feita bit a bit, cadenciada pelo sinal de relógio *TXclk* fornecido pelo software, garantindo sincronização entre o hardware e o recetor. O módulo serializa uma palavra de 4 bits (código da tecla) num formato de trama com bit de início (*start bit*), bits de dados e bit de fim (*stop bit*), totalizando 8 transições de relógio por transmissão.

No pedido de transmissão: O *Ring Buffer* verifica se o *Key Transmitter* está disponível através do sinal *KBfree/CTS*. Caso esteja, envia o código da tecla e ativa o sinal *Load/Wreg*, indicando que os dados são válidos. O *Key Transmitter* carrega então o código no seu registo interno e inicia a transmissão, baixando o sinal *KBfree* para sinalizar que está ocupado. O *Ring Buffer* observa *KBfree* a 0 e baixa o sinal *Load*, libertando o barramento. A transmissão decorre bit a bit, cadenciada por *TXclk*, até ao fim da trama.

No fim da transmissão: Ao atingir o último bit da trama, o contador interno do módulo gera o sinal *TC* (*Terminal Count*), indicando que a transmissão está concluída. O módulo volta ao estado de espera, reativando *KBfree* para sinalizar ao *Ring Buffer* que está pronto para uma nova transmissão. Se existirem mais teclas na fila, o ciclo repete-se imediatamente.

O módulo *Key Transmitter* integra 3 componentes: *i) Registo de dados*, responsável por armazenar o código da tecla recebido do *Ring Buffer* e disponibilizá-lo para serialização; *ii) Contador e MUX de serialização*, que seleciona sequencialmente cada bit a transmitir, incluindo o *start bit* e o *stop bit*, cadenciado por *TXclk*; *iii) Key Transmitter Control*, a máquina de estados que sincroniza a comunicação com o *Ring Buffer* e controla o fluxo de transmissão série.

O módulo foi construído conforme apresentado no diagrama da Figura 12.

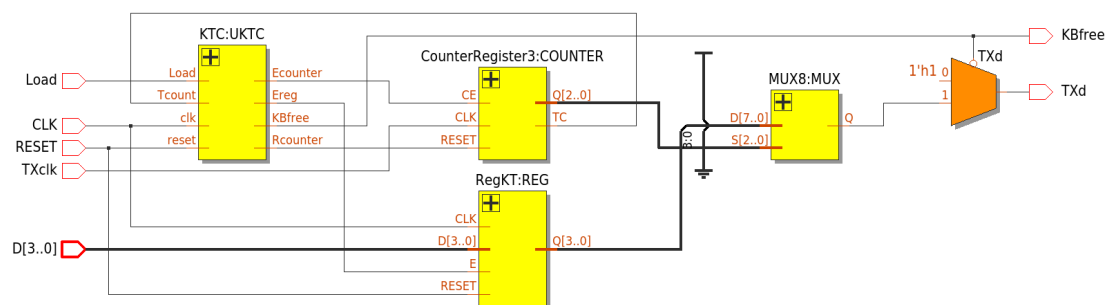


Figura 12: Diagrama de blocos do *Key Transmitter*

i) Datapath - Protocolo de transmissão

Registo

O registo de 4 bits é o bloco responsável por gravar o código da tecla após ler do *Ring Buffer*. É ativado pela máquina de estados através do sinal *Load/EReg*

CounterRegister O contador/registo de 3 bits guarda o estado atual de contagem. A contagem

de valor n corresponde ao n ésimo bit da trama a ser transmitido. A trama possui 8 bits: o bit de inicialização (0), o *Start*, os bits do código da tecla, o *Stop* e o bit de repouso (1), tal como descreve a figura 13 Logo o armazenador de contagem necessita de ser de 3 bits.

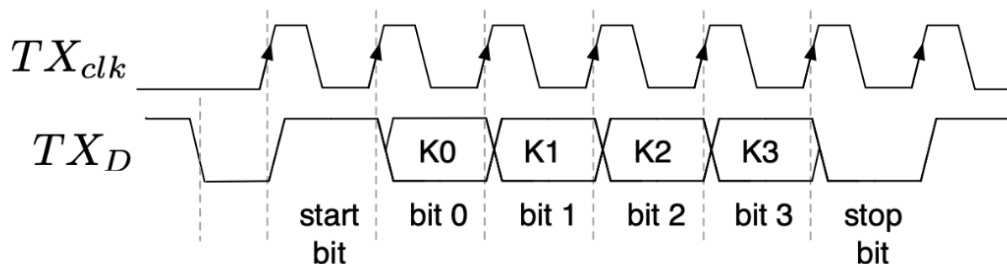


Figura 13: Diagrama temporal do protocolo de transmissão

MUX8:1 Por fim, o multiplexer de 8 bits irá "descodificar" o valor de contagem e selecionar o bit de trama correto a enviar. O multiplexer garante que o protocolo é inicializado pelo bit a nível 0, e mantém TXd a 1 quando está em repouso. Adicionalmente, o seu valor de saída é comparado ao $KBfree$ para garantir que o bit foi enviado enquanto o protocolo está a correr. Caso não esteja, o TXd mantém o seu valor padrão (1).

ii) Key Transmitter Control

O *Key Transmitter Control* atua como o bloco lógico central do módulo transmissor. Trata-se de uma máquina de estados desenhada especificamente para coordenar a conversão de dados do formato paralelo para o formato série, garantindo uma interface de comunicação fiável com o *Ring Buffer*.

A principal responsabilidade deste bloco é gerir o protocolo de sincronização com o sistema a montante e comandar os componentes internos do transmissor. Através das suas saídas lógicas, o KTC controla os momentos exatos em que o registo deve capturar uma nova palavra de dados e quando o contador interno deve iniciar e terminar a contagem dos bits enviados pela linha de comunicação. Esta ASM é implementada de forma comportamental, com um total de 2 estados lógicos: *i) WAITINGDATA*, preparação da nova informação; *ii) SENDDATA*, que coloca o protocolo em funcionamento.

Estado 0 - WAITINGDATA

Este é o estado inicial para o qual a máquina arranca e para onde regressa sempre que não há atividade. O seu objetivo é preparar o sistema para uma nova receção de dados e sinalizar disponibilidade. O sinal $KBfree$ é colocado a 1, para o *Ring Buffer* saber que o transmissor está livre. Em simultâneo, o sinal $Rcounter$ é ativado a 1, forçando o contador de bits a permanecer limpo (em reset) para que a próxima transmissão comece a contar a partir de zero. O KTC monitoriza continuamente o sinal de entrada *Load* (proveniente do *Ring Buffer*). Enquanto este for 0, a máquina permanece em WAITINGDATA. Assim que o *Load* sobe para 1 (indicando que há uma nova tecla disponível), o KTC ativa o sinal $Ereg$ a 1. O *Enable Register* faz com que

o *Shift Register* engula o código da tecla no imediato. No flanco de relógio seguinte, a ASM transita para o estado 1 (SENDDATA).

Estado 1 - SENDDATA

Este estado representa o ciclo de trabalho do módulo. Aqui, a receção de novos dados é bloqueada e o foco passa exclusivamente para a serialização e envio da informação para o exterior (*Software*). A primeira ação ao entrar neste estado é baixar o sinal *KBfree* para 0, informando o *Ring Buffer* de que o transmissor se encontra ocupado. O sinal *Rcounter* é desativado para libertar o contador, e o sinal *Ecounter* é ativado a 1. Esta ativação liberta o contador para começar a contar os ciclos de relógio e autoriza o deslocamento sequencial dos bits na linha de saída. A ASM fica retida no estado SENDDATA a coordenar a transmissão até receber a confirmação de que o processo terminou. Essa confirmação chega através do sinal *Tcount* (*Terminal Count*), gerado pelo contador interno quando atinge o número limite de bits da trama. Quando o KTC deteta o *Tcount* a 1, reconhece o fim da transmissão e, no flanco de relógio seguinte, retorna de imediato ao estado WAITINGDATA, ficando pronto para repetir todo o ciclo.

A figura 14 apresenta o diagrama do Algoritmo da Máquina de Estados do Key Transmitter Control.

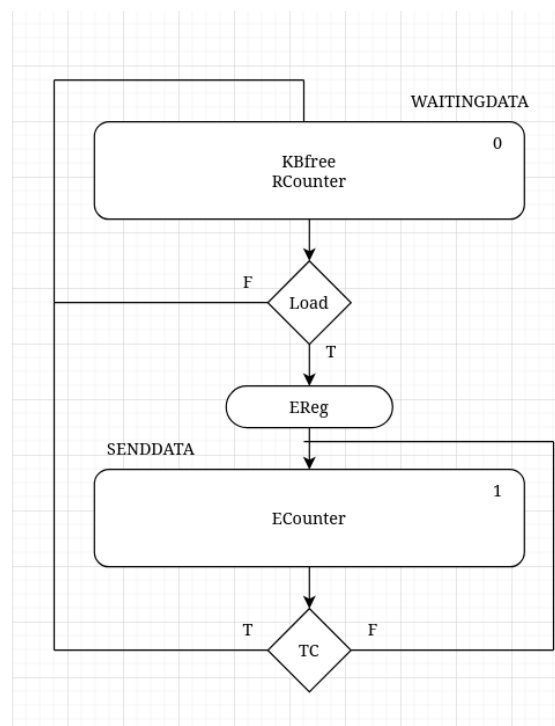


Figura 14: Algoritmo da Máquina de Estados do *Key Transmitter*

iii) *Key Transmitter* - Vista Geral

O *Key Transmitter* é o módulo encarregue de receber os códigos de tecla em formato paralelo (provenientes do *Ring Buffer*) e convertê-los numa sequência de dados em série para serem transmitidos de forma síncrona para o sistema central. No seu estado de repouso, o módulo indica que está disponível para iniciar uma operação através da ativação do sinal *KBfree* a

1. Quando o *Ring Buffer* sinaliza a existência de dados válidos através do sinal *Load*, o *Key Transmitter* captura o código da tecla de 4 bits para o seu registo e coloca o sinal *KBfree* a 0, assumindo o estado de ocupado e bloqueando o fluxo de dados à entrada. Imediatamente a seguir, o módulo inicia o protocolo de comunicação série: o software verifica o bit de inicialização para receber os bits de dados e contruir a trama desde então. Após esta verificação, o software gera impulsos de *TXclk* e o Multiplexer vai enviando os bits á medida. Este processo de envio é monitorizado continuamente por um contador de bits interno. Assim que este contador atinge o limite da trama e ativa o sinal *Tcount* (*Terminal Count*), indicando que toda a informação foi transmitida com sucesso, a geração do relógio *TXclk* é interrompida, *TXd* regressa ao estado inativo a 1 e o sinal *KBfree* é reposto a 1. O módulo conclui assim a serialização e regressa ao estado de repouso inicial, ficando novamente disponível para ler a próxima tecla guardada no *Ring buffer*.

2.1.4 Vista geral do funcionamento

O Keyboard Reader é o módulo responsável por estabelecer a interface completa entre um teclado matricial (4x4) e o sistema de processamento central da Ticket Machine. A sua principal missão é detetar interações físicas do utilizador, codificá-las digitalmente, gerir o armazenamento temporário dessa informação e garantir a sua entrega fiável através de um protocolo de comunicação série. Para assegurar um funcionamento robusto e sem perda de dados (mesmo perante velocidades de escrita rápidas), a arquitetura deste sistema está dividida em três estágios sequenciais: o *Key Decode*, o *Ring Buffer* e o *Key Transmitter*. O ciclo de operação inicia-se na frente do sistema, com o ***Key Decode***. Este módulo passa a maior parte do tempo num estado de varrimento contínuo, monitorizando ativamente as linhas e colunas do teclado. Quando o utilizador pressiona uma tecla, o varrimento é imediatamente interrompido e o módulo gera o código de 4 bits correspondente a essa coordenada física. Para passar a informação adiante, o *Key Decode* ativa um sinal de validação (*Kval*), informando o estágio seguinte de que há um dado pronto a ser guardado. Esse estágio intermédio é o ***Ring Buffer***, que atua como uma fila de espera (FIFO). Ao detetar o sinal do *Key Decode*, o *buffer* verifica a sua memória RAM interna. Se houver espaço disponível, captura o código da tecla e responde com um sinal de reconhecimento (*Kack*). Este passo de confirmação é crucial: ao assumir a guarda do dado, o *Ring Buffer* liberta o *Key Decode* de imediato, permitindo que este regresse ao varrimento para detetar a próxima tecla, independentemente de o sistema central estar ou não pronto para receber a informação anterior. Por fim, a transmissão dos dados armazenados fica a cargo do ***Key Transmitter***. Quando este módulo se encontra livre de transmissões anteriores (sinalizando a sua disponibilidade via *KBfree*), o *Ring Buffer* transfere-lhe a tecla mais antiga guardada na memória de forma paralela. O transmissor guarda esse código no seu registo interno, assume o estado de ocupado e inicia a serialização da informação. Avisa o recetor externo e o software gera os pulsos de relógio (*TXclk*) necessários para deslocar os bits do código sequencialmente através da linha de dados (*TXd*).

Quando a trama é totalmente enviada, o *Key Transmitter* conclui o processo e declara-se novamente disponível. Em suma, a integração coordenada destes três blocos permite que o *Keyboard Reader* absorva eventos físicos de forma rápida, os ordene cronologicamente em memória e os

despache de forma síncrona e ritmada, garantindo a sincronização entre a lentidão humana e o tempo de máquina.

Este módulo foi construído de acordo com o diagrama da figura 15

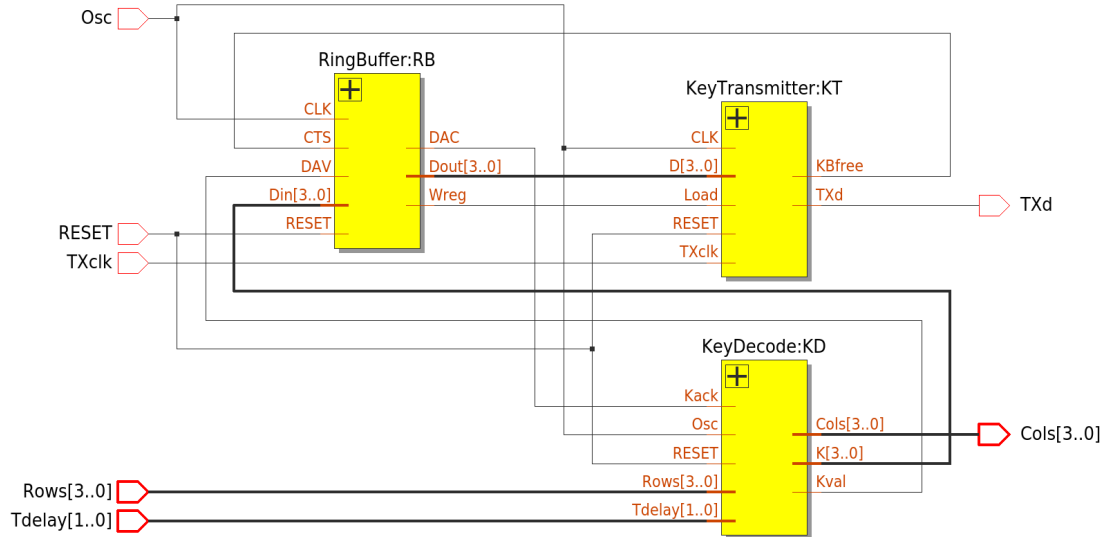


Figura 15: Diagrama de blocos do módulo *Keyboard Reader*

2.2 Coin Acceptor

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, o valor monetário da moeda inserida é codificado em $Coin_{id_{2:0}}$, conforme a codificação apresentada na Tabela 1. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 16b. O *Control* pode devolver as moedas inseridas no moedeiro através da ativação do sinal *eject* durante dois segundos, ou recolher as moedas presentes no moedeiro através da ativação do sinal *collect* durante dois segundos. Este módulo é simulado através dos interruptores e LEDs da FPGA programável.

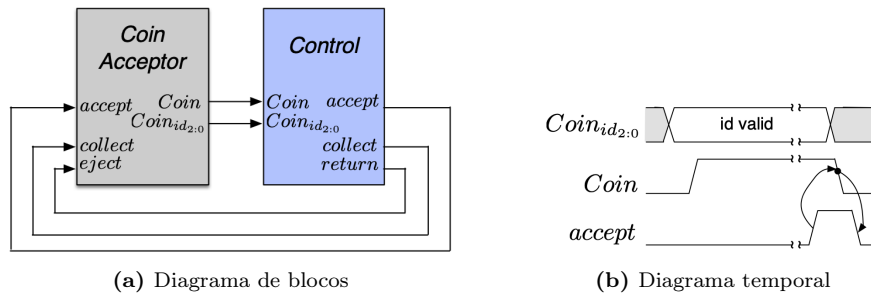


Figura 16: Bloco *Coin Acceptor*

Tabela 1: Codificação do valor facial das moedas

Moeda [€]	Codificação [$Cid_2 Cid_1 Cid_0$]
0,05	000
0,10	001
0,20	010
0,50	011
1,00	100
2,00	101

2.3 Port Expander LCD

O módulo *Port Expander LCD* [2] (*PELCD*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 17.

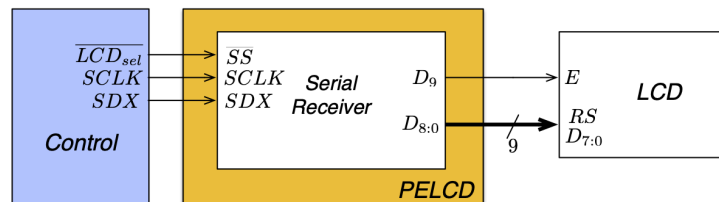


Figura 17: Diagrama de blocos do módulo *Port Expander LCD* (*PELCD*)

O LCD opera através de um conjunto de sinais de controlo e de um barramento de dados: **RS (Register Select)**: determina o tipo de informação transmitida. Se $RS = 0$: o valor presente no barramento corresponde a um comando; Se $RS = 1$: o valor presente no barramento corresponde a um dado a apresentar no ecrã. **E (Enable)**: sinal de validação da transferência. O LCD apenas lê os dados presentes nas entradas quando ocorre uma transição ativa deste sinal. **DATA[7:0]**: barramento de 8 bits utilizado para transmitir comandos ou dados.

O PELCD recebe do *Software* um conjunto de sinais que identificam o dado a transmitir e o respetivo tipo (comando ou carácter). Quando um novo valor é recebido, o módulo coloca o byte no barramento de dados e configura adequadamente o sinal RS. De seguida é gerado um pulso no sinal Enable (E), permitindo que o LCD capture a informação presente nas entradas. Após a conclusão da transferência, o sinal Enable regressa ao estado inativo e o módulo fica preparado para uma nova operação.

Os comandos são utilizados para configurar ou controlar o funcionamento do LCD. Alguns exemplos são:

- Limpeza do ecrã; - Posicionamento do cursor; - Ativação ou desativação do cursor; - Configuração do modo de funcionamento do display.

Nestes casos o sinal RS é colocado a 0 e o valor do comando é enviado através do barramento de dados.

Para apresentar texto no visor, os caracteres são enviados um a um para o LCD. Neste modo o sinal RS é colocado a 1 e cada byte corresponde ao código ASCII do carácter a apresentar. Após cada transferência o LCD avança automaticamente a posição do cursor, permitindo a escrita sequencial de palavras e frases. Este módulo integra 1 componente: *i) Serial Receiver*, que vai receber serialmente os dados enviados pelo emissor (*Software*).

2.3.1 Serial Receiver

O módulo **Serial Receiver** é responsável por receber informação enviada serialmente pelo sistema e disponibilizá-la aos periféricos de destino. Este módulo recebe as tramas transmitidas pelo *Serial Emitter* e encaminha os dados para o LCD ou para o mecanismo de impressão de bilhetes. Este bloco integra: *i) Shift Register*, que recebe dados em série; *ii) Hold Register*, recebe a data do registo de deslocamento.

i) Shift Register

O Shift Register é responsável por armazenar temporariamente os bits recebidos. Sempre que chega um novo bit: - O conteúdo atual é deslocado uma posição; - O novo bit é inserido na extremidade do registo; - Após a receção de todos os bits da trama, o registo contém a palavra completa.

Este mecanismo permite converter informação recebida sequencialmente numa palavra paralela que pode ser utilizada pelos restantes módulos.

ii) Hold Register

Em seguida, os bits correspondentes aos dados são armazenados num registo próprio. Este registo disponibiliza a informação em paralelo para o periférico selecionado.

Resumindo:

- O **ShiftRegisterPELCD** recebe os bits série transmitidos pelo emissor (SDX) sincronizados com SCLK; - Enquanto SS está ativo (LOW), os bits são deslocados para dentro do registo; - Quando a transmissão termina, a subida de SS carrega o conteúdo do Shift Register para o **Hold Register**; - O Hold Register mantém a trama estável à saída (Q) até à receção da próxima mensagem.

2.4 Port Expander Ticket Dispenser

O módulo *Port Expander Ticket Dispenser* [3] (*PETD*) implementa a interface com o mecanismo de impressão de bilhetes (*Ticket Dispenser*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mecanismo, conforme representado na Figura 18.

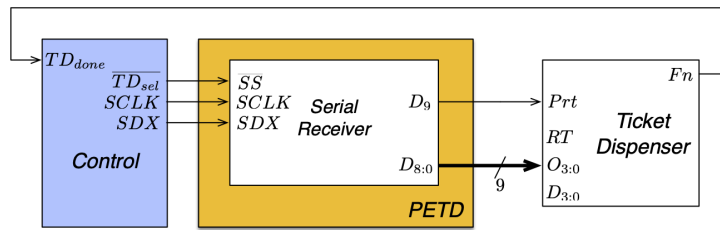


Figura 18: Diagrama de blocos do módulo *Port Expander Ticket Dispenser (PETD)*

Este módulo é responsável pela interface entre o sistema digital e o mecanismo de impressão de bilhetes. A sua função consiste em receber os dados provenientes do Serial Receiver, armazená-los temporariamente e gerar os sinais de controlo necessários para iniciar o processo de impressão. O PETD recebe uma palavra de dados em paralelo contendo a informação necessária para a emissão do bilhete. Esta informação inclui:

- Tipo de viagem (Roundtrip);
- Estação de origem;
- Estação de destino;
- Comandos de controlo associados ao processo de impressão (Prt).

Após a receção dos dados, o módulo gera um sinal de ativação destinado ao mecanismo de impressão. Assim como o PELCD, este módulo integra um componente, o Serial Receiver. 2.3.1

2.5 Control

O Módulo de Controlo constitui a camada de inteligência e gestão da Ticket Machine, implementada em Kotlin, que traduz os requisitos funcionais do sistema em operações lógicas executáveis. Ao contrário da camada de hardware, que lida com a sinalização elétrica e a sincronização temporal, este software funciona como o sistema operativo da máquina, orquestrando a interação entre o utilizador, os periféricos e os processos internos. A implementação do módulo *Control* foi realizada em software, seguindo a arquitetura lógica apresentada na Figura 19.

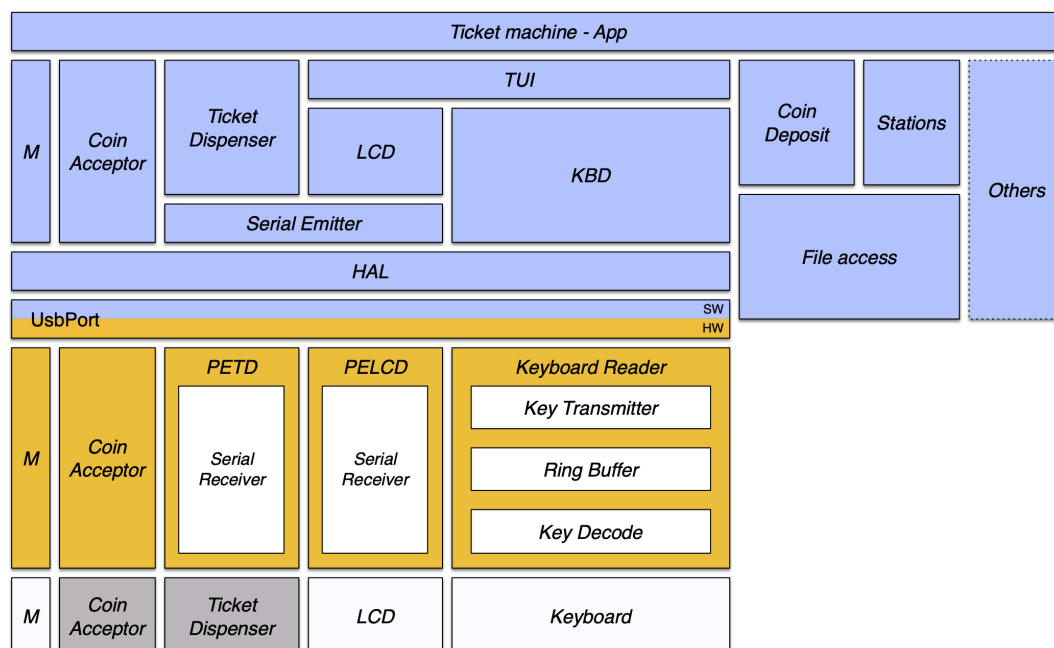


Figura 19: Diagrama lógico da Máquina de Venda de Bilhetes (*Ticket Machine*)

2.5.1 Hardware Abstract Layer

É o módulo base que encapsula o acesso direto às portas de entrada e saída da FPGA (através da classe *UsbPort*). A sua função é fornecer métodos genéricos (*readBits*, *setBits*, *clrBits*, *isBit*) que mascaram a complexidade das operações de bits, garantindo que o resto do software não precise de interagir diretamente com o barramento USB.

2.5.2 KBD

Este módulo é responsável por gerir a interface com o teclado matricial. Utiliza o *KeyReceiver* para obter os códigos de tecla crus e converte-os nos caracteres correspondentes (ex: '0'-'9', '*', '#', 'A'-'D'). Disponibiliza métodos como *waitKey*, que permite ao sistema aguardar de forma bloqueante por uma entrada do utilizador, sendo essencial para a interação no menu.

2.5.3 Key Receiver

Implementa o protocolo de comunicação série de baixo nível para receber o código de uma tecla pressionada. Gere a temporização do sinal de relógio (*TXclk*) e a leitura da linha de dados (*TXD*), construindo a trama de bits recebida e validando-a para isolar o código da tecla, garantindo que o KBD receba dados íntegros.

2.5.4 LCD

Este módulo é a interface para o ecrã alfanumérico. Gere a inicialização e o envio de comandos e dados. Utiliza o *SerialEmitter* para transmitir a informação por protocolo série ao *Port Expander* do LCD. Inclui métodos para controlo de cursor, limpeza de ecrã e posicionamento de caracteres, abstraindo a lógica de comandos específica do controlador.

2.5.5 Ticket Dispenser

Coordena a comunicação com o hardware de impressão/dispensa de bilhetes. Através do `SerialEmitter`, envia os dados da viagem (tipo de bilhete, origem, destino) para o periférico de impressão. Garante que os sinais de controlo (como o sinal `Prt`) sejam ativados e desativados corretamente para evitar bloqueios no sistema.

2.5.6 Serial Emitter

Este módulo é o coração da comunicação síncrona. É responsável por gerir o protocolo que envia 10 bits de dados para periféricos (LCD ou Ticket Dispenser). Gere o sinal de *Slave Select* (SS) e o relógio de transmissão (SCLK), garantindo que os dados são enviados com a temporização correta para cada periférico selecionado.

2.5.7 Coin Acceptor

Monitoriza o hardware de aceitação de moedas. Lê os sinais de identificação (CoinID) que indicam o valor da moeda inserida. Providencia métodos para aceitar (`coinAccept`), recolher (`coinCollect`) ou devolver (`coinReturn`) a moeda, garantindo que o hardware reagir corretamente às decisões do utilizador ou do sistema.

2.5.8 Coin Deposit

Gera a lógica contabilística das moedas inseridas. Mantém o estado do saldo atual (`deposited`), processa a inserção de moedas, devolve em caso de cancelamento e gere a contagem de moedas armazenadas. Integra com o `FileAccess` para persistir o estado financeiro da máquina.

2.5.9 M

Um módulo simples que monitoriza o estado do interruptor de manutenção (M). Ao ler o bit de estado correspondente, permite que a aplicação detete se a máquina deve entrar em modo de configuração/manutenção ou operar em modo normal de venda, funcionando como um sensor de estado global.

2.5.10 Stations

Este módulo gere a tabela de estações disponíveis para venda. Carrega a informação das estações a partir do ficheiro de configuração, mantém o estado de bilhetes vendidos por estação (`sold`) e coordena a escrita dessas atualizações no ficheiro de persistência. É fundamental para a lógica de negócio do `TicketMachineApp`.

2.5.11 File Access

Este módulo é responsável por toda a interface entre a aplicação e o sistema de ficheiros local. A sua função é garantir que dados críticos — como o estado dos contadores de vendas das estações e o inventário de moedas — não sejam perdidos após o encerramento da máquina. O `FileAccess` encapsula a lógica de leitura e escrita de ficheiros de texto, utilizando estruturas de dados

(*Data Classes*) que facilitam o mapeamento direto entre o conteúdo dos ficheiros (*stations.txt*, *CoinDeposit.txt*) e os objetos manipulados em memória durante o tempo de execução.

2.5.12 Text User Interface

O módulo TUI atua como a camada de apresentação de alto nível da aplicação. A sua função é traduzir eventos e estados do sistema em mensagens formatadas e visualmente organizadas para o ecrã LCD. Ao contrário dos drivers de baixo nível, o TUI não comunica diretamente com o hardware, mas utiliza os métodos do driver LCD para realizar operações complexas, como a centralização de texto, o posicionamento dinâmico de strings e a representação de estados de transação.

2.5.13 Ticket Machine APP

Este é o ponto de entrada e o "cérebro" da aplicação. O *TicketMachineApp* orquestra o fluxo de execução global, contendo a máquina de estados principal que define o comportamento da máquina de venda de bilhetes. É neste módulo que a lógica de negócio é implementada: a transição entre o modo de venda (seleção de destino, inserção de moedas e impressão) e o modo de manutenção, a gestão dos tempos de espera (*timeout*) e a integração entre os inputs do utilizador (via KBD) e as saídas informativas (via TUI e LCD). Este módulo atua, assim, como o integrador final, coordenando a colaboração entre todos os drivers de periféricos para garantir uma experiência de utilizador fluida e funcional.

3 Resultados Experimentais

Para validar o funcionamento do circuito recorreu-se ao uso de ficheiros de teste. Cada módulo em hardware foi testado em simulação com um ficheiro gerador de estímulos. Tratando dos módulos mais abrangentes:

Key Decode: Para validar este bloco temos de assegurar se o varrimento, e principalmente a unidade de controlo, estão bem sincronizados entre si e com os restantes módulos. Isto inclui: Observar se o *Key Scan* faz o varrimento, detetar a pressão de uma tecla, a existência de comunicação (*Kscan*, *Kack*), a transição de estados e, na segunda fase, o mecanismo de repeat com os respetivos intervalos configuráveis;

Ring Buffer: Para validar este bloco é necessário garantir que ele comunica bem com o *Key Decode* e o *Key Transmitter*, visto ser o bloco intermediário. Isto inclui validar o gestor de endereços e o bloco de controlo. A melhor forma de testar este bloco é encher a sua memória interna com 16 códigos e, em seguida, recorrendo a estímulos ou até mesmo com o próprio *Key Transmitter*, esvaziar tudo em seguida. Isto permite observar todos os estados e sinais de saída, como o estado da memória (*Full/Empty*), o ramo de escrita, o ramo de leitura, o conteúdo da RAM, a comunicação (*DAV/DAC*), a seleção de endereços e a liberação dos códigos;

Key Transmitter: Para validar este bloco precisamos assegurar se comunica bem com o *Ring Buffer*, ié, indicação de que está ocupado/desocupado (*KBfree*), e que o protocolo de transmissão está correto (0, Start, K, Stop, 1). Para testar este módulo é necessário que a testbench substitua o Ring Buffer de modo a que receba as teclas. Uma alternativa melhor é agrupar os dois módulos em uma testbench. Assim todos os testes são validados de uma vez; **KeyBoard Reader:** Como este bloco agrupa os 3 (e com os subblocos isoladamente testados), este teste consiste mais em analisar e validar a sincronização entre módulos.

Port Expanders: Partilham ambos o mesmo bloco, o *SerialReceiver*. Para validar este bloco precisamos primeiro de assegurar que os registos fazem as suas funções. Em seguida o bloco é simulado com uma testbench que assume o papel de software, e verificar a saída de dados.

Já o software é testado com recurso ao Hardware emulado ou com a FPGA. Cada ficheiro de software contém uma testbench integrada:

Hardware Abstract Layer: Foi necessário implementar as lógicas de leitura e escrita nos portos do *UsbPort*. A testbench realiza todas as operações e o porto é observável no emulador ou na FPGA;

Key Receiver: Implementa um recetor em série. Gera os pulsos de *TXclk* de modo a receber a trama enviada pelo transmissor. A testbench disponibiliza o código binária das teclas;

KBD: Integrando o *Key Receiver*, este módulo converte os códigos binários em caracteres para melhor leitura, conjuntamente com uma função auxiliar que aguarda uma tecla válida. A testbench tem a capacidade de testar o módulo *Keyboard Reader* por completo, sendo capaz de despejar as teclas armazenadas no Ring Buffer na sua execução, o mecanismo de repetição com diferentes intervalos, entre outros;

LCD: Este módulo integra funções de envio de comandos e de dados. Requer uma sequência de inicialização específica. A testbench permite registar dados no display. Permite também validar o funcionamento do PELCD pela FPGA, visto que este é o recetor dos dados e o mesmo que os envia para o LCD físico;

File Access / Stations / CoinDeposit: Estes módulos envolvem manipulação de ficheiros. A testbench é capaz de ler os dados dos ficheiros, executar as operações de venda, limpeza, aceitação, cancelamento, e de registar tudo de volta;

M: Sendo o módulo mais simples, trata de ler a chave de entrada para o modo de Manutenção. A testbench lê o estado da chave em tempo real.

Coin Acceptor: Gera os sinais de aceitação, coleta e ejeção, assim como lê uma possível inserção e o ID da moeda. A testbench aguarda pela inserção por parte do utilizador e é capaz de realizar as 3 tarefas;

Serial Emitter: Responsável por enviar os dados para os periféricos. O módulo trata de construir a trama e envia para o Port Expander do endereço selecionado. A testbench permite validar a emissão da trama, assim como o recetor em hardware (*Serial Receiver*);

Ticket Dispenser: Responsável por enviar a trama para dispensar um bilhete. Integra o Serial Emitter.; ***Text User Interface:*** Este módulo trata das interfaces observáveis no display, desde o ecrã inicial, o ecrã de seleção da estação, o ecrã de pagamento, o ecrã de manutenção, entre outros. Integra o LCD e o KBD. Assim, é possível observar algum efeitos do teclado no display. A testbench realiza 3 testes: O teste do teclado, o teste de seleção e a exibição de interfaces;

Ticket Machine APP: Sendo o módulo mais abrangente e a aplicação final, este módulo integra todos os previamente referidos, orquestrando tudo em conjunto. Implementa o Modo de Manutenção e o Modo de Venda.

O Software e o Hardware foram testados isoladamente e, em seguida, conjuntamente com a placa programável FPGA. Isto permite testar todos os módulos possíveis, seja individualmente para garantir a lógica correta, seja em conjunto, para garantir que não há problemas de sincronização.

4 Conclusões

O trabalho realizado consistiu na conceção, implementação e validação de um sistema completo para uma máquina de venda de bilhetes, integrando módulos de interface de utilizador, gestão de transações e controlo de hardware. O projeto foi desenvolvido através de uma abordagem modular em VHDL, focada na fiabilidade do processamento digital e na eficiência da comunicação entre subsistemas. A solução abrangeu o desenvolvimento do *Keyboard Reader* — assegurando a captura robusta de inputs e a transmissão série —, do bloco de gestão de moedas e do controlador de interface visual via *Port Expander*. A integração destes componentes foi testada na placa de desenvolvimento FPGA DE10-Lite, onde a implementação das máquinas de estados permitiu uma coordenação precisa entre os diferentes protocolos de comunicação, garantindo que o fluxo de dados entre a entrada de moedas, a seleção do destino e a resposta do LCD ocorresse de forma sincronizada e sem colisões. Os resultados experimentais confirmam o funcionamento do sistema de acordo com as especificações funcionais e os requisitos de interface definidos. A validação demonstrou que a arquitetura proposta é capaz de processar as interações do utilizador em tempo real, gerir corretamente o estado da máquina de venda e apresentar a informação de forma legível no ecrã, mesmo sob condições de utilização intensiva. Em suma, o projeto demonstrou a eficácia da decomposição modular na resolução de problemas complexos de hardware, resultando numa implementação robusta, escalável e fiel aos princípios de design de sistemas digitais aprendidos.

5 Trabalho Futuro

O trabalho pode ser melhorado através da implementação de uma gestão de trocos mais complexa, integrando a validação de diferentes tipos de moeda em simultâneo. Adicionalmente, seria possível adicionar um sistema de 'timeout' que cancelaria automaticamente uma transação iniciada caso o utilizador não complete a seleção no LCD num determinado período, aumentando a eficiência da máquina.

O trabalho pode ser melhorado com a implementação de um mecanismo de filtragem de ruído (debouncing) mais sofisticado no módulo de Key Decode, recorrendo a filtros digitais adaptativos. Outra melhoria seria a expansão da interface visual no LCD para incluir menus dinâmicos de navegação, permitindo ao utilizador consultar o histórico de vendas ou ajustar configurações de preço diretamente na interface.

Referências

- [1] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *KeyBoardReader datasheet*, 2026, ISEL.
- [2] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander LCD datasheet*, 2026, ISEL.
- [3] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander Ticket Dispenser datasheet*, 2026, ISEL.
- [4] Terasic *DE10-lite User Manual*, 2020, Terasic.
- [5] Terasic *de10-Lite Exp. Board - Reference Manual*, 2022, Pedro Miguens Matutino

A Ticket Machine - Hardware

A.1 TicketMachine.vhd

```

-----
-- TICKET MACHINE
-----

library ieee;
use IEEE.std_logic_1164.all;

entity TicketMachine is
    port
    (
        --Inputs
        M: in std_logic;                --Chave de manutençao
        CoinId: in std_logic_vector(2 downto 0);    --Identificador da moeda
        Coin: in std_logic;            --Chave de inserçao de moeda
        Rows: in std_logic_vector(3 downto 0);    --Linhas do teclado
        RESET: in std_logic;            --Clear da maquina
        Tdelay: in std_logic_vector(1 downto 0);    --Intervalo de tempo de repeat
        Osc: in std_logic;                --Relogio da FPGA
        CT: in std_logic;                --Chave da colheita do bilhete

        --Outputs
        Cols: out std_logic_vector(3 downto 0);    --Colunas do teclado
        D9: out std_logic;                --MSB Data out (PELCD)
        D: out std_logic_vector(8 downto 0);    --9 LSB Data out (PELCD)
        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5: out STD_LOGIC_VECTOR(7 downto 0); --Displays de 7 segmentos
        Collect: out std_logic;            --Sinal de recolha por parte da entidade consumidora
        Eject: out std_logic;            --Sinal de devolução por parte da entidade consumidora
        Accept: out std_logic;            --Sinal de aceitação por parte da entidade consumidora
        Prt: out std_logic);            --Sinal de impressao do bilhete
end TicketMachine;

architecture arch_TM of TicketMachine is
    --Leitor do teclado
    component KeyboardReader is
        port(
            Rows: in std_logic_vector(3 downto 0);
            Tdelay: in std_logic_vector(1 downto 0);
            RESET: in std_logic;
            Osc: in std_logic;
            TXclk: in std_logic;
            TXd: out std_logic;

```

```

    Cols: out std_logic_vector(3 downto 0));
end component;
--Usbport
component UsbPort IS
    PORT
    (
        inputPort:  IN  STD_LOGIC_VECTOR(7 DOWNT0 0);
        outputPort :  OUT  STD_LOGIC_VECTOR(7 DOWNT0 0)
    );
END component;
--Port Expander Liquid Cristal Display
component PELCD is
    port
    (
        SDX: in std_logic;
        SCLK: in std_logic;
        SS: in std_logic;
        RESET: in std_logic;
        D9: out std_logic;
        D: out std_logic_vector(8 downto 0));
end component;
--Ticket Dispenser
component TICKET_DISPENSER is
    port ( RT, Prt, CollectTicket: in STD_LOGIC;
        O, D: in STD_LOGIC_VECTOR(3 downto 0);
        Fn: out STD_LOGIC;
        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5: out STD_LOGIC_VECTOR(7 downto 0) );
end component;
--Port Expander Ticket Dispenser
component PETD is
    port
    (
        SDX: in std_logic;
        SCLK: in std_logic;
        SS: in std_logic;
        RESET: in std_logic;
        D9: out std_logic;
        D: out std_logic_vector(8 downto 0));
end component;

--Sinais intermedios
signal I0, I1, I2, I3, I4, I5, I6, I7: std_logic; --Porto de entrada
signal O0, O1, O2, O3, O4, O5, O6, O7: std_logic; --Porto de saida

```

```
signal Done: std_logic;          --Finish do Ticket Dispenser
signal Output_usb: std_logic_vector(7 downto 0); --Porto de saida
signal D9_out: std_logic;        --MSB de data (PELCD)
signal D_out: std_logic_vector(8 downto 0);   --Bits restantes de data (PELCD)

--Instanciação
begin
--Sinais de entrada
I0 <= CoinId(0);
I1 <= CoinId(1);
I2 <= CoinId(2);
I3 <= Coin;
I6 <= M;

--Sinais do porto de saida
O0 <= Output_usb(0);
O1 <= Output_usb(1);
O2 <= Output_usb(2);
O3 <= Output_usb(3);
O4 <= Output_usb(4);
O5 <= Output_usb(5);
O6 <= Output_usb(6);
O7 <= Output_usb(7);

--Sinais de saida
Collect <= O6;
Eject <= O5;
Accept <= O4;
Prt <= D9_out;

KR: KeyboardReader port map( --Instanciação do Leitor de teclado
    Rows => Rows,
    RESET => RESET,
    Tdelay => Tdelay,
    Osc => Osc,
    TXclk => O7,
    TXd => I7,
    Cols => Cols);

SRLCD: PELCD port map( --Instanciação do PELCD
    SDX => O0,
    SCLK => O1,
    SS => O2,
```

```

RESET => RESET,
D9 => D9,
D => D);

SRTD: PETD port map(      --Instanciação do PETD
    SDX => 00,
    SCLK => 01,
    SS => 03,
    RESET => RESET,
    D9 => D9_out,
    D => D_out);

TD: TICKET_DISPENSER port map( --Instanciação do Ticket Dispenser
    RT => D_out(0),
    Prt => D9_out,
    CollectTicket => CT,
    0(0) => D_out(1),
    0(1) => D_out(2),
    0(2) => D_out(3),
    0(3) => D_out(4),
    D(0) => D_out(5),
    D(1) => D_out(6),
    D(2) => D_out(7),
    D(3) => D_out(8),
    Fn => Done,
    HEX0 => HEX0,
    HEX1 => HEX1,
    HEX2 => HEX2,
    HEX3 => HEX3,
    HEX4 => HEX4,
    HEX5 => HEX5);

UUSBPORT: UsbPort port map( --Instanciação do UsbPort
    inputPort(0) => I0,
    inputPort(1) => I1,
    inputPort(2) => I2,
    inputPort(3) => I3,
    inputPort(4) => Done,
    inputPort(5) => '0',
    inputPort(6) => I6,
    inputPort(7) => I7,
    outputPort => Output_usb);

```

```
end arch_TM;
```

```
-----  
-- PINOS  
-----
```

```
--M: SW[4]  
--CoinId: SW[2..0]  
--Coin: SW[3]  
--Tdelay: SW[6..5]  
--CT: SW[8]  
--RESET: SW[9]  
  
--Collect: LEDR[2]  
--Eject: LEDR[1]  
--Accept: LEDR[0]  
--Prt: LEDR[9]
```

A.2 Atribuição de Pinos

B Ticket Machine - Software (*Control*)

B.1 Ticket Machine - App

```
import TUI.write  
import isel.leic.utils.Time  
import kotlin.system.exitProcess  
  
//=====   
//                                TICKET MACHINE  
//=====   
object TicketMachineApp {  
    /**  
     * Origin station (protected)  
     *  
     * Key timeout (protected)  
     */  
    private const val ORIGIN_STATION = 0  
    private const val WAIT_KEY_MS = 2000L  
  
    /**  
     * Function: maintenanceMode()  
     *  
     * Description: Maintenance mode of the application
```

```

* @param void
* @return void
*/
private fun maintenanceMode() {
    val options = listOf(
        "#-Print Ticket",
        "A-Station Cnt.",
        "B-Coins Cnt.",
        "C-Reset Cnt.",
        "D-Shutdown"
    )
    var optIdx = 0
    /**
     * Function: printTicket_M()
     *
     * Description: Simulates ticket printing
     * @param void
     * @return void
     * @see KBD.waitKey
     * @see TUI.printTicket
     * @see M.enabled
     * @see TUI.toPrint
     * @see TUI.processing
     * @see TicketDispenser.lowerPrt
     * @see TicketDispenser.activatePrintingTicket
     * @see TUI.collectFinished
     * @see TUI.collectTicket
     * @see TUI.vendingAborted
     */
    fun printTicket_M() {
        var idx = 0
        var roundTrip = true
        val stations = Stations.stations
        if (stations.isEmpty()) return
        while (M.enabled()) {
            val price = stations[idx].price * if (roundTrip) 2 else 1
            TUI.printTicket(stations[idx].station, idx, price / 100.0)
            when (val key = KBD.waitKey(WAIT_KEY_MS)) {
                'A' -> idx = if (idx > 0) idx - 1 else stations.size - 1
                'B' -> idx = if (idx < stations.size - 1) idx + 1 else 0
                'C' -> roundTrip = !roundTrip
                '*' -> {
                    val st = stations[idx]

```

```

    TUI.toPrint(st.station, !roundTrip)
    if (KBD.waitKey(7000L) != '*') {
        TUI.vendingAborted()
        Time.sleep(1200)
        continue
    }
    TUI.processing(st.station)
    Time.sleep(3000)
    TicketDispenser.activatePrintingTicket(roundTrip, ORIGIN_STATION, idx)
    TUI.collectTicket(st.station)
    while (!HAL.isBit(INPUTPORTS.FN.mask)) {
        Time.sleep(10)
    }
    TicketDispenser.lowerPrt(roundTrip, ORIGIN_STATION, idx)
    Time.sleep(200)
    TUI.collectFinished()
    Time.sleep(1500)
    return
}
'#' -> {
    TUI.vendingAborted()
    Time.sleep(1200)
    return
}
KBD.none -> continue
in '0'..'9' -> {
    val digit = key.digitToInt()
    val nextIdx = (idx % 10) * 10 + digit
    if (nextIdx < stations.size) {
        idx = nextIdx
    } else if (digit < stations.size) {
        idx = digit
    }
}
}
}

/**
 * Function: stationCnt()
 *
 * Description: Displays counters for stations
 * @param void
 * @return void

```



```

* @see KBD.waitKey
* @see M.enabled
* @see TUI.stationCount
*/
fun stationCnt() {
    var idx = 0
    val stations = Stations.stations
    if (stations.isEmpty()) return
    while (M.enabled()) {
        TUI.stationCount(stations[idx].station, idx, stations[idx].sold)
        when (val key = KBD.waitKey(WAIT_KEY_MS)) {
            'A' -> idx = if (idx > 0) idx - 1 else stations.size - 1
            'B' -> idx = if (idx < stations.size - 1) idx + 1 else 0
            '#' -> return
            KBD.none -> continue
            in '0'..'9' -> {
                val digit = key.digitToInt()
                val nextIdx = (idx % 10) * 10 + digit
                if (nextIdx < stations.size) {
                    idx = nextIdx
                } else if (digit < stations.size) {
                    idx = digit
                }
            }
        }
    }
}

/**
 * Function: coinsCnt()
 *
 * Description: Display counters for coins
 * @param void
 * @return void
 * @see KBD.waitKey
 * @see TUI.printCoins
 * @see M.enabled
 * @see CoinDeposit.getCount
 */
fun coinsCnt() {
    val values = intArrayOf(5, 10, 20, 50, 100, 200)
    var idx = 0
    while (M.enabled()) {
        TUI.printCoins(values[idx] / 100.0, CoinDeposit.getCount(idx), idx)
    }
}

```

```

        when (val key = KBD.waitKey(WAIT_KEY_MS)) {
            'A' -> idx = if (idx > 0) idx - 1 else values.size - 1
            'B' -> idx = if (idx < values.size - 1) idx + 1 else 0
            '#' -> return
            KBD.none -> return
            in '0'..'9' -> {
                val digit = key.digitToInt()
                if (digit in values.indices) idx = digit
            }
        }
    }
}

/**
 * Function: resetCounters()
 *
 * Description: Resets counters
 * @param void
 * @return void
 * @see TUI.yesOrNo
 * @see TUI.write
 * @see Stations.reset
 * @see CoinDeposit.resetCnt
 */
fun resetCounters() {
    if (TUI.yesOrNo("Reset Counters?", 7000)) {
        TUI.write("Resetting...", 0, 0, true, true)
        Stations.reset()
        CoinDeposit.resetCnt()
        Time.sleep(2000)
        TUI.write("Done!", 0, 0, true, true)
        Time.sleep(5000)
    }
}

/**
 * Function: shutdown()
 *
 * Description: Saves everything and closes process
 * @param void
 * @return void
 * @see Stations.save
 * @see CoinDeposit.saveCoins
 * @see TUI.write
 */

```

```

    fun shutDown() {
        Stations.save()
        CoinDeposit.saveCoins()
        TUI.write("Shutting down...", 0, 0, true, true)
        Time.sleep(1000)
        LCD.clear()
        exitProcess(0)
    }
    while (M.enabled()) {
        TUI.maintenanceScreen(options[optIdx])
        when (KBD.waitKey(WAIT_KEY_MS)) {
            '#' -> printTicket_M()
            'A' -> stationCnt()
            'B' -> coinsCnt()
            'C' -> resetCounters()
            'D' -> if (TUI.yesOrNo("Shutdown?", 5000L)) shutDown()
            KBD.none -> optIdx = (optIdx + 1) % options.size
        }
    }
}

/**
 * Function: normalMode()
 *
 * Description: Vending mode of the application
 * @param void
 * @return void
 */
private fun normalMode() {
    val stations = Stations.stations
    if (stations.isEmpty()) return
    /**
     * Function: pollCoin()
     *
     * Description: Manages the hardware (Coin acceptor) and processes acceptance of new coin
     * Updates the deposit
     * @param void
     * @return void
     * @see CoinAcceptor.coinInserted
     * @see CoinAcceptor.coinAccept
     * @see CoinDeposit.insert
     */
    fun pollCoin() {
        if (!CoinAcceptor.coinInserted()) return

```

```

        val id = CoinAcceptor.getCoinId()
        if (id in 0..5) {
            CoinDeposit.insert(id)
        }
        CoinAcceptor.coinAccept()
    }
}

/**
 * Function: abortVending()
 *
 * Description: Cancels the current transaction. Returns inserted coins.
 * Updates the deposit
 * @param void
 * @return void
 * @see CoinAcceptor.coinReturn
 * @see CoinDeposit.cancel
 * @see CoinDeposit.getTotal
 * @see TUI.vendingAborted
 * @see TUI.vendingAborted2
 */
fun abortVending() {
    val returned = CoinDeposit.getTotal()
    if (returned > 0) {
        CoinAcceptor.coinReturn()
        CoinDeposit.cancel()
        TUI.vendingAborted2(returned / 100.0)
    } else {
        TUI.vendingAborted()
    }
    Time.sleep(1200)
}

/**
 * Function: waitForPayment()
 *
 * Description: Waits for the insertion of the value by the user
 * @param station Destination
 * @param required Price
 * @return Boolean
 * @see Others.centsToEuros
 * @see TUI.write
 * @see pollCoin
 * @see M.enabled
 * @see CoinDeposit.getTotal
 * @see KBD.waitKey

```

```

*/
fun waitForPayment(station: String, required: Int): Boolean {
    var lastInserted = -1
    while (!M.enabled()) {
        pollCoin()
        val inserted = CoinDeposit.getTotal()
        if (inserted != lastInserted) {
            TUI.write(station, 0, 0, true, true)
            TUI.write(
                "${6.toChar()}${Utils.centsToEuros(inserted)}/${Utils.centsToEuros(required)}",
                1,
                0,
                true,
                false
            )
            lastInserted = inserted
        }
        if (inserted >= required) return true
        if (KBD.waitKey(200) == '#') return false
    }
    return false
}

var idx = 0
var roundTrip = false
var redraw = true
while (!M.enabled()) {
    val st = stations[idx]
    val price = st.price
    pollCoin()
    TUI.printTicket2(st.station, idx, roundTrip, price / 100.0)
    when (val key = KBD.waitKey(WAIT_KEY_MS)) {
        'A' -> idx = if (idx > 0) idx - 1 else stations.size - 1
        'B' -> idx = if (idx < stations.size - 1) idx + 1 else 0
        'C' -> roundTrip = !roundTrip
        '*' -> {
            TUI.toPrint(st.station, !roundTrip)
            if (KBD.waitKey(7000L) != '*') {
                abortVending()
                redraw = true
                continue
            }
            if (!waitForPayment(st.station, price)) {
                abortVending()
            }
        }
    }
}

```

```

        continue
    }
    if (!CoinDeposit.exchange(price)) {
        abortVending()
        continue
    }
    CoinAcceptor.coinCollect()
    TUI.processing(st.station)
    Time.sleep(3000)
    TicketDispenser.activatePrintingTicket(roundTrip, ORIGIN_STATION, idx)
    TUI.collectTicket(st.station)
    while (!HAL.isBit(INPUTPORTS.FN.mask)) {
        Thread.sleep(10)
    }
    TicketDispenser.lowerPrt(roundTrip, ORIGIN_STATION, idx)
    Stations.sold(st.station)
    Stations.save()
    TUI.collectFinished()
    Time.sleep(1500)
    return
}
'#' -> {
    abortVending()
    return
}
KBD.none -> continue
in '0'..'9' -> {
    val digit = key.digitToInt()
    val nextIdx = (idx % 10) * 10 + digit
    if (nextIdx < stations.size) {
        idx = nextIdx
    } else if (digit < stations.size) {
        idx = digit
    }
}
}
}

/**
 * Function: init()
 *
 * Description: This functions inits the object
 * @param void

```

```

* @return void
* @see M.init
* @see CoinDeposit.init
* @see CoinAcceptor.init
* @see TicketDispenser.init
* @see TUI.init
* @see Stations.init
*/
fun init() {
    M.init()
    CoinAcceptor.init()
    TicketDispenser.init()
    TUI.init()
    CoinDeposit.init()
    Stations.init()
}
/**
 * Function: Program()
 *
 * Description: The core of the APP
 * @param void
 * @return void
 * @see TUI.startMenu
 * @see M.enabled
 * @see maintenanceMode
 * @see normalMode
 * @see KBD.waitKey
 */
fun program() {
    var scrollIdx = 0
    var lastScrollTime = 0L
    val scrollSpeedMs = 350L
    val welcomeText = "${6.toChar()}Press # to Enter${7.toChar()}           ${6.toChar()}Press M
    TUI.startMenu()
    Time.sleep(1500)
    while (true) {
        if (M.enabled()) {
            maintenanceMode()
            TUI.startMenu()
            scrollIdx = 0
            lastScrollTime = 0L
        }
        val currentTime = System.currentTimeMillis()

```

```

        if (currentTime - lastScrollTime >= scrollSpeedMs) {
            Miscellaneous.writeScrollingText(welcomeText, 1, scrollIdx)
            scrollIdx++
            lastScrollTime = currentTime
        }
        val key = KBD.waitKey(20L)
        if (key == '#') {
            normalMode()
            TUI.startMenu()
            Time.sleep(3000)
            scrollIdx = 0
            lastScrollTime = 0L
        }
    }
}

//=====
//                                     MAIN
//=====
fun main() {
    TicketMachineApp.init()

    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    write("Powering", 0, 0, false, false)
    for (i in 1..6) {
        val blocks = "${3.toChar()}.repeat(i) + "${4.toChar()}.repeat(6 - i)
        write(blocks, 0, 9, false, false)
        Time.sleep(500)
    }
    repeat(3) { Time.sleep(600); print("") }
    println("${Miscellaneous.RESET}\n")

    println("${Miscellaneous.GREEN}===== Navigation in Vending Mode =====")
    println("${Miscellaneous.YELLOW}#${Miscellaneous.RESET} -> Enter vending mode")
    println("Inside:")
    println("  ${Miscellaneous.YELLOW}C${Miscellaneous.RESET} -> Select trip type (One-way/Round-")
    println("  ${Miscellaneous.YELLOW}*${Miscellaneous.RESET} -> Confirm Selection")
    println("  ${Miscellaneous.RED}#${Miscellaneous.RESET} -> Abort / Return Coins")

    println("\n${Miscellaneous.GREEN}===== Navigation in Maintenance Mode =====")
    println("  ${Miscellaneous.YELLOW}A${Miscellaneous.RESET} -> Stations Cnt   | ${Miscellaneous")
    println("  ${Miscellaneous.YELLOW}C${Miscellaneous.RESET} -> Reset Cnt       | ${Miscellaneous

```



```

    TicketMachineApp.program()
}

```

B.2 M

```

import isel.leic.utils.Time
//=====
//
//=====
object M {
    /**
     * Function: enabled()
     *
     * Description: Returns the logic value of the maintenance key
     * @param void
     * @return Boolean - Logic value of the bit of interest
     * @see HAL.isBit
     */
    fun enabled() = HAL.isBit(INPUTPORTS.M_OUT.mask)
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see HAL.init
     */
    fun init() {
        HAL.init()
    }
}
//=====
//
//=====
fun main() {
    M.init()
    TUI.init()
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press the M button")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} On FPGA, use Switch 4")
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
}

```

```

print("\n")
Time.sleep(1000)
println("=====")
print("\r${Miscellaneous.RED}${Miscellaneous.BOLD} M State: OFF")
var prev = M.enabled()
while (true) {
    val curr = M.enabled()
    val state = if (M.enabled()) "ON" else "OFF"
    val stateColor = if (M.enabled()) Miscellaneous.GREEN else Miscellaneous.RED
    if (M.enabled() != prev) {
        print("\r$stateColor M State: $state")
        TUI.write("M State: $state", 0, 0, true, true)
        prev = curr
    }
    Time.sleep(10)
}
}

```

B.3 TUI

```

import isel.leic.utils.Time
import java.util.Date
import java.text.SimpleDateFormat

//=====
//                                     TEXT USER INTERFACE
//=====

object TUI {
    private val dateFormatter = SimpleDateFormat("dd/MM/yyyy HH:mm")
    /**
     * Protected Function: center()
     *
     * Description: Calculates the starting column required to center a text on the LCD
     * @param text Text to be centered
     * @return Starting column position
     */
    private fun center(text: String): Int {
        val getCol = (LCD.COLS - text.length) / 2
        return if (getCol < 0) 0 else getCol
    }
    /**
     * Function: write()
     *

```

```

    * Description: Improved writing function. Writes text to the LCD with optional centering and
    * @param text Text to be displayed
    * @param lin LCD line position
    * @param col LCD column position
    * @param center Enables text centering
    * @param clear Clears the LCD before writing
    * @return void
    * @see LCD.clear
    * @see LCD.cursor
    * @see LCD.write
    */
fun write(text: String, lin: Int, col: Int, center: Boolean, clear: Boolean) {
    if (clear) LCD.clear()
    if (text.length <= LCD.COLS) {
        val startCol = if (center) center(text) else col
        LCD.cursor(lin, startCol)
        LCD.write(text)
    } else {
        val line0Text = text.substring(0, LCD.COLS)
        val startCol0 = if (center) center(line0Text) else 0
        LCD.cursor(0, startCol0)
        LCD.write(line0Text)
        val remaining = text.substring(LCD.COLS)
        val line1Text = if (remaining.length > LCD.COLS) remaining.substring(0, LCD.COLS) else remaining
        val startCol1 = if (center) center(line1Text) else 0
        LCD.cursor(1, startCol1)
        LCD.write(line1Text)
    }
}

/**
 * Function: init()
 *
 * Description: Initializes the LCD and Keyboard modules
 * @param void
 * @return void
 * @see LCD
 * @see KBD
 */
fun init() {
    LCD.init()
    KBD.init()
    LCD.hideCursor()
}

```

```

/**
 * Function: startMenu()
 *
 * Description: Displays the system home screen with current date and time
 * @param void
 * @return void
 * @see SimpleDateFormat
 */
fun startMenu() {
    write("${5.toChar()}ByteTrip!${5.toChar()}", 0, 0, true, true)
    val dateStr = dateFormatter.format(Date())
    write(dateStr, 1, 0, true, false)
}

/**
 * Function: maintenanceScreen()
 *
 * Description: Displays the maintenance screen with the selected option
 * @param option Maintenance option to display
 * @return void
 */
fun maintenanceScreen(option: String) {
    write("${6.toChar()}Maintenance${7.toChar()}", 0, 0, true, true)
    write(option, 1, 0, true, false)
}

/**
 * Function: writeKeyOnLCD()
 *
 * Description: Writes a pressed key to the LCD
 * @param key Character to display
 * @return void
 * @see LCD.write
 */
fun writeKeyOnLCD(key: Char) {
    LCD.write(key.toString())
}

/**
 * Function: yesOrNo()
 *
 * Description: Displays a confirmation screen and waits for user input
 * @param text Question to display
 * @param time Timeout in milliseconds
 * @return True if '*' is pressed, false otherwise
 * @see KBD.waitKey

```

```

*/
fun yesOrNo(text: String, time: Long): Boolean {
    write(text, 0, 0, true, true)
    write("*>Yes    Other>No", 1, 0, true, false)
    return KBD.waitKey(time) == '*'
}
/**
 * Function: vendingAborted()
 *
 * Description: Displays a vending aborted message
 * @param void
 * @return void
 */
fun vendingAborted() {
    write("Vending Aborted.", 0, 0, true, true)
}
/**
 * Function: vendingAborted2()
 *
 * Description: Displays a vending aborted message and returned amount
 * @param price Amount returned to the user
 * @return void
 */
fun vendingAborted2(price: Double) {
    write("Vending Aborted.", 0, 0, true, true)
    write("Returned ${"%0.2f".format(price)}${0.toChar()}", 1, 0, true, false)
}
/**
 * Function: collectTicket()
 *
 * Description: Displays ticket collection instruction
 * @param station Destination station name
 * @return void
 */
fun collectTicket(station: String) {
    write(station, 0, 0, true, true)
    write("${6.toChar()}Collect Ticket${7.toChar()}", 1, 0, true, false)
}
/**
 * Function: collectFinished()
 *
 * Description: Displays a thank-you message after ticket collection
 * @param void

```

```

    * @return void
    */
fun collectFinished() {
    write("Thank You!", 0, 0, true, true)
    write("Have a nice trip", 1, 0, true, false)
}
/**
 * Function: toPrint()
 *
 * Description: Displays ticket printing confirmation screen
 * @param station Destination station name
 * @param rt Indicates if the ticket is round trip
 * @return void
 */
fun toPrint(station: String, rt: Boolean) {
    write(station, 0, 0, true, true)
    if (rt) write("${1.toChar()} * > To Print", 1, 0, true, false)
    if (!rt) write("${1.toChar()}${2.toChar()} * > To Print", 1, 0, true, false)
}
/**
 * Function: printTicket()
 *
 * Description: Displays ticket information before printing
 * @param station Destination station name
 * @param keyCode Station selection code
 * @param price Ticket price
 * @return void
 */
fun printTicket(station: String, keyCode: Int, price: Double) {
    write(station, 0, 0, true, true)
    write("${"%02d".format(keyCode)}${1.toChar()}${2.toChar()}          ${"%02f".format(price)}$
        , 1, 0, true, false)
}
/**
 * Function: printTicket2()
 *
 * Description: Displays ticket information including round-trip calculation
 * @param station Destination station name
 * @param rt Indicates if the ticket is round trip
 * @param price Base ticket price
 * @return void
 */
fun printTicket2(station: String, keyCode: Int, rt: Boolean, price: Double) {

```

```

        val price2 = price * if (rt) 2 else 1
        write(station, 0, 0, true, true)
        if (rt) write("${"%02d".format(keyCode)}${1.toChar()}${2.toChar()}           ${"%2f".format(
        if (!rt) write("${"%02d".format(keyCode)}${1.toChar()}           ${"%2f".format(price2)}${0
    }
/**
 * Function: processing()
 *
 * Description: Displays a processing message
 * @param station Destination station name
 * @return void
 */
fun processing(station: String) {
    write(station, 0, 0, true, true)
    write("Processing", 1, 0, false, false)
    for (i in 1..3) {
        val blocks = "${3.toChar()}.repeat(i) + "${4.toChar()}.repeat(3 - i)
        write(blocks, 1, 12, false, false)
        Time.sleep(1000)
    }
}
/**
 * Function: printCoins()
 *
 * Description: Displays coin information in maintenance mode
 * @param value Coin value
 * @param amount Number of available coins
 * @param code Coin identifier code
 * @return void
 */
fun printCoins(value: Double, amount: Int, code: Int) {
    write("${"%2f".format(value)}${0.toChar()}", 0, 0, true, true)
    write("${"%02d".format(code)}${1.toChar()}${2.toChar()}           $amount",
        1, 0, false, false)
}
/**
 * Function: stationCount()
 *
 * Description: Displays station ticket sales information
 * @param station Station name
 * @param idx Station identifier code
 * @param count Number of tickets sold
 * @return void

```

```

    */
    fun stationCount(station: String, idx: Int, count: Int) {
        write(station, 0, 0, true, true)
        write("${"%02d".format(idx)}${1.toChar()}${2.toChar()}          $count"
            , 1, 0, false, false)
    }
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    TUI.init()
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Select to skip to desired test")
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Keyboard Test -> 0\n Y/N Test -> 1\n Int")
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("===== ${Miscellaneous.CYAN}${Miscellaneous.BOLD} Keyboard Test")
    while (true) {
        TUI.startMenu()
        val testSelect = KBD.waitKey(100000)
        when (testSelect) {
            '0' -> {
                println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press C to clear display")
                println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press D to quit")
                println("=====")
                TUI.write("Test1: Keyboard", 0, 0, true, true)
                Time.sleep(1000)
                TUI.write("Initializing...", 1, 0, true, false)
                Time.sleep(3000)
                LCD.clear()
                TUI.write("> ", 0, 0, false, true)
                while (true) {
                    val key = KBD.getKey()
                    if (key != KBD.none) {
                        when (key) {
                            'D' -> break
                            'C' -> {
                                LCD.clear()
                            }
                        }
                    }
                }
            }
        }
    }
}

```



```

        TUI.write("> ", 0, 0, false, true)
    }
    else -> TUI.writeKeyOnLCD(key)
}
}
}
}
}
'1' -> {
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press D to quit")
    println("=====")
    TUI.write("Test2: Yes or No", 0, 0, true, true)
    Time.sleep(1000)
    TUI.write("Initializing...", 1, 0, true, false)
    Time.sleep(3000)
    LCD.clear()
    val prompts = arrayOf("LEIC melhor curso?", "Cereais primeiro?", "Messi > CR7")
    for (i in prompts) {
        if (KBD.getKey() == 'D') break
        val yN = TUI.yesOrNo(i, 5000)
        TUI.write(if (yN) "You choose: Yes" else "You choose: No", 0, 0, true, true)
        Time.sleep(2000)
    }
}
'2' -> {
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press D to quit")
    println("=====")
    TUI.write("Interfaces", 0, 0, true, true)
    Time.sleep(1000)
    TUI.write("Initializing...", 1, 0, true, false)
    Time.sleep(3000)
    LCD.clear()
    TUI.write("Use the keyboard to navigate", 0, 0, true, true)
    while (true) {
        val key = KBD.getKey()
        when (key) {
            '0' -> TUI.printTicket("Maldivas", 13, 4.55)
            '1' -> TUI.vendingAborted()
            '2' -> TUI.vendingAborted2(0.67)
            '3' -> TUI.collectTicket("Roma")
            '4' -> TUI.collectFinished()
            '5' -> TUI.toPrint("Atenas", true)
            '6' -> TUI.processing("Troia")
            '7' -> TUI.printCoins(2.55, 4, 8)
        }
    }
}

```

```

        '8' -> TUI.stationCount("Wakanda", 7, 2)
        '9' -> TUI.printTicket2("Caraibas", 16, false, 2.25)
        'D' -> break
    }
}
}
}
}
}
}
}

```

B.4 Coin Acceptor

```
import isel.leic.utils.Time
```

```

//=====
//                                     COIN ACCEPTOR
//=====
object CoinAcceptor {
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see HAL.isBit
     */
    fun init() {
        HAL.init()
    }
    /**
     * Function: getCoinId()
     *
     * Description: Reads CoinID input bits
     * @param void
     * @return id0 or (id1 shl 1) or (id2 shl 2) - Frame with full coin id
     * @see HAL.isBit
     */
    fun getCoinId(): Int {
        val id0 = if (HAL.isBit(INPUTPORTS.COIN_ID0.mask)) 1 else 0
        val id1 = if (HAL.isBit(INPUTPORTS.COIN_ID1.mask)) 1 else 0
        val id2 = if (HAL.isBit(INPUTPORTS.COIN_ID2.mask)) 1 else 0
        return id0 or (id1 shl 1) or (id2 shl 2)
    }
}

```

```

/**
 * Function: coinValue()
 *
 * Description: Gets coin value from ID
 * @param void
 * @return Coin value associated with id or null when id is invalid
 * @see getCoinId
 */
fun coinValue(): Int? {
    val value = when (getCoinId()) {
        0 -> 5
        1 -> 10
        2 -> 20
        3 -> 50
        4 -> 100
        5 -> 200
        else -> null
    }
    return value
}

/**
 * Function: coinInserted()
 *
 * Description: Checks if a coin has been inserted
 * @param void
 * @return HAL.isBit(INPUTPORTS.COIN.mask) - Logic value of Coin
 * @see HAL.isBit
 */
fun coinInserted() = HAL.isBit(INPUTPORTS.COIN.mask)

/**
 * Function: coinAccept()
 *
 * Description: Accepts the coin inserted
 * @param void
 * @return void
 * @see HAL.setBits
 * @see HAL.clrBits
 */
fun coinAccept() {
    if (!coinInserted()) return
    HAL.setBits(OUTPUTPORTS.ACCEPT.mask)
    while (HAL.isBit(INPUTPORTS.COIN.mask)) {Time.sleep(1)}
    HAL.clrBits(OUTPUTPORTS.ACCEPT.mask)
}

```

```

}
/**
 * Function: coinCollect()
 *
 * Description: Collects the coin
 * @param void
 * @return void
 * @see HAL.setBits
 * @see HAL.clrBits
 */
fun coinCollect() {
    HAL.setBits(OUTPUTPORTS.COLLECT.mask)
    Time.sleep(2000)
    HAL.clrBits(OUTPUTPORTS.COLLECT.mask)
}
/**
 * Function: coinReturn()
 *
 * Description: Ejects the coin
 * @param void
 * @return void
 * @see HAL.setBits
 * @see HAL.clrBits
 */
/**Ejects the coin**/
fun coinReturn() {
    HAL.setBits(OUTPUTPORTS.EJECT.mask)
    Time.sleep(2000)
    HAL.clrBits(OUTPUTPORTS.EJECT.mask)
}
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    CoinAcceptor.init()
    println("${Miscellaneous.RED}${Miscellaneous.BOLD} Requires FPGA")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} To select a coin, use Switches 2..0")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} To insert a coin, use Switch 3\n")
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
}

```

```

Time.sleep(1000)
print("\n")
Time.sleep(1000)
println("=====")
println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Choose a task:")
println("0 - Accept")
println("1 - Collect")
println("2 - Eject")
println("3 - Quit")
println("=====")
var quit = false
while (!quit) {
    val isInserted = CoinAcceptor.coinInserted()
    val currentId = if (isInserted) CoinAcceptor.getCoinId() else "None"
    val currentValue = CoinAcceptor.coinValue()?.let { Utils.centsToEuros(it) } ?: "N/A"
    print("\r${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Status: Inserted: $isInserted | ID: ")
    print("\r${Miscellaneous.YELLOW}${Miscellaneous.BOLD}> ")
    val key = readln().toInt()
    when (key) {
        0 -> {
            if (isInserted) {
                println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Accepting coin...")
                CoinAcceptor.coinAccept()
            } else {
                println("${Miscellaneous.RED}${Miscellaneous.BOLD} No coin detected to ac")
            }
        }
        1 -> {
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Collecting coin...")
            CoinAcceptor.coinCollect()
        }
        2 -> {
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Ejecting coin...")
            CoinAcceptor.coinReturn()
        }
        3 -> {
            quit = true
            println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Quitting...")
        }
    }
}
Time.sleep(100)
}

```

B.5 Coin Deposit

```
import isel.leic.utils.Time
```

```
//=====
//
//=====
object CoinDeposit {
    private var deposited = 0
    private var coins = arrayOf<FileAccess.Coins>()
    private var currCoins = IntArray(6)
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see FileAccess.readCoins
     */
    fun init() {
        coins = FileAccess.readCoins("CoinDeposit.txt")
    }
    /**
     * Function: insert()
     *
     * Description: Inserts coin on deposit
     * @param id Coin identification
     * @return void
     */
    fun insert(id: Int) {
        val value = when (id) {
            0 -> 5
            1 -> 10
            2 -> 20
            3 -> 50
            4 -> 100
            5 -> 200
            else -> 0
        }
        if (value > 0) {
            deposited += value
            currCoins[id]++
        }
    }
}
```

```

}
/**
 * Function: exchange()
 *
 * Description: Aborts & returns coins
 * @param value Value to be exchanged
 * @return Boolean - Mainly to check if deposit has enough to exchange
 */
fun exchange(value: Int): Boolean {
    if (deposited < value) return false
    for (i in currCoins.indices) {
        coins[i].amount += currCoins[i]
    }
    currCoins = IntArray(6)
    deposited = 0
    return true
}
/**
 * Function: cancel()
 *
 * Description: Aborts & returns
 * @param void
 * @return void
 */
fun cancel() {
    deposited = 0
    currCoins = IntArray(6)
}
/**
 * Function: getTotal()
 *
 * Description: Returns total deposited
 * @param void
 * @return deposited - Value on the deposit
 */
fun getTotal() = deposited
/**
 * Function: getCount()
 *
 * Description: Returns coin amount
 * @param id Coin identification
 * @return coins[id].amount - Amount of the coin type
 */

```

```

fun getCount(id: Int) = coins[id].amount
/**
 * Function: resetCnt()
 *
 * Description: Resets counters
 * @param void
 * @return void
 */
fun resetCnt() {
    for (i in coins.indices) {
        coins[i].amount = 0
    }
}
/**
 * Function: saveCoins()
 *
 * Description: Saves coins
 * @param void
 * @return void
 */
fun saveCoins() {
    FileAccess.writeCoins("CoinDeposit.txt", coins)
}
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    CoinDeposit.init()
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====")
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Available Coins (IDs): 0:5c,\n1:10c,\n2:")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD}=====")
    println(" Choose a task:")
    println("0..5 - Insert Coin")
    println("6 - Exchange")
    println("7 - Cancel")

```



```
println("8 - Save")
println("9 - Reset Counters")
println("10 - Quit")
println("=====")
var quit = false
while (!quit) {
    print("\r${Miscellaneous.GREEN}${Miscellaneous.BOLD} Current Deposit: ${CoinDeposit.getTo
    print("\r${Miscellaneous.GREEN}${Miscellaneous.BOLD}> ")
    val key = readln().toInt()
    when (key) {
        in 0..5 -> {
            val id = key
            CoinDeposit.insert(id)
            println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Inserted coin ID: $id.
        }
        6 -> {
            print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Amount to exchange for? "
            val amount = readln().toIntOrNull() ?: 0
            if (CoinDeposit.exchange(amount)) println("${Miscellaneous.GREEN}${Miscellaneous
            else println("${Miscellaneous.RED}${Miscellaneous.BOLD} Exchange failed! Depo
        }
        7 -> {
            CoinDeposit.cancel()
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Task cancelled. Deposit
        }
        8 -> {
            CoinDeposit.saveCoins()
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Coins saved to file.")
        }
        9 -> {
            CoinDeposit.resetCnt()
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Counters reset.")
        }
        10 -> {
            quit = true
            println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Quitting...")
        }
    }
}
Time.sleep(100)
}
```

B.6 Stations

```
import isel.leic.utils.Time
```

```
//=====
//
//=====
object Stations {
    var stations = arrayOf<FileAccess.Stations>()
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see FileAccess.readStations
     */
    fun init() {
        stations = FileAccess.readStations("stations.txt")
    }
    /**
     * Function: sold()
     *
     * Description: Increments sold whenever a ticket is sold
     * @param station Destination
     * @return void
     */
    fun sold(station: String) {
        for (i in stations) {
            if (i.station == station) {
                i.sold++
            }
        }
    }
    /**
     * Function: save()
     *
     * Description: Saves on file
     * @param void
     * @return void
     * @see FileAccess.writeStations
     */
    fun save() {
```

```

        FileAccess.writeStations("stations.txt", stations)
    }
    /**
     * Function: reset()
     *
     * Description: Resets counters
     * @param void
     * @return void
     */
    fun reset() {
        for (i in stations) {
            i.sold = 0
        }
    }
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    Stations.init()
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====")
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Available Stations:${Miscellaneous.RESET}")
    Stations.stations.forEach { println("${Miscellaneous.GREEN}- ${it.station} (Price: ${it.price}
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD}=====")
    print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Insert station for testing: ")
    val s = readln()
    println("\n${Miscellaneous.GREEN}${Miscellaneous.BOLD} Station '$s' selected.")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Choose a task:")
    println("0 - Sell a ticket")
    println("1 - Reset counters")
    println("2 - Save")
    println("3 - Quit")
    println("=====")
    var exit = false
    while (!exit) {
        val station = Stations.stations.find { it.station == s }

```

```

    if (station == null) {
        print("${Miscellaneous.RED}${Miscellaneous.BOLD} Error: Station '$s' not found!${Miscellaneous.RESET}")
        break
    }
    print("\r${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Station: ${station.station} | Sold: ${station.sold} ${Miscellaneous.RESET}")
    print("\r> ")
    val key = readln().toInt()
    when (key) {
        0 -> {
            Stations.sold(s)
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Ticket for $s sold!")
        }
        1 -> {
            Stations.reset()
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Counters have been reset!")
        }
        2 -> {
            Stations.save()
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Saved!")
        }
        3 -> {
            println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Quitting...")
            exit = true
        }
    }
    Time.sleep(100)
}
}

```

B.7 File Access

```

import isel.leic.utils.Time
import java.io.BufferedReader
import java.io.FileReader
import java.io.PrintWriter

```

```

//=====
//                                     File Access
//=====

object FileAccess {
    /**Data structures**/
    data class Stations(val price: Int, var sold: Int, val station: String)
}

```

```
data class Coins(val value: Int, var amount: Int)

/**
 * Function: createReader()
 *
 * Description: Creates a file reader
 * @param fileName Name of the file to read
 * @return BufferedReader(FileReader(fileName))
 * @see BufferedReader
 */
private fun createReader(fileName: String): BufferedReader {
    return BufferedReader(FileReader(fileName))
}

/**
 * Function: createWriter()
 *
 * Description: Creates a file writer
 * @param fileName Name of the file to read
 * @return PrintWriter(fileName)
 * @see PrintWriter
 */
private fun createWriter(fileName: String): PrintWriter {
    return PrintWriter(fileName)
}

/**
 * Function: toStations()
 *
 * Description: Converts a text line into a Stations object
 * @param line Text line in the format "price;sold;station"
 * @return Stations object
 * @see Stations
 */
fun toStations(line: String): Stations {
    val parts = line.split(";")
    return Stations(
        parts[0].toInt(),
        parts[1].toInt(),
        parts[2]
    )
}

/**
 * Function: toCoins()
 *
 * Description: Converts a text line into a Coins object

```

```

    * @param line Text line in the format "value;amount"
    * @return Coins object
    * @see Coins
    */
fun toCoins(line: String): Coins {
    val parts = line.split(";")
    return Coins(
        parts[0].toInt(),
        parts[1].toInt()
    )
}

/**
 * Function: readStations()
 *
 * Description: Reads all stations from a file and stores them in an array
 * @param fileName Name of the file to read
 * @return Array of Stations
 * @see Stations
 */
fun readStations(fileName: String): Array<Stations> {
    val readFile = createReader(fileName)
    var line = readFile.readLine()
    var stations = mutableListOf<Stations>()
    while (line != null) {
        stations.add(toStations(line))
        line = readFile.readLine()
    }
    return stations.toTypedArray()
}

/**
 * Function: readCoins()
 *
 * Description: Reads all coins from a file and stores them in an array
 * @param fileName Name of the file to read
 * @return Array of Coins
 * @see Coins
 */
fun readCoins(fileName: String): Array<Coins> {
    val readFile = createReader(fileName)
    var line = readFile.readLine()
    var coins = mutableListOf<Coins>()
    while (line != null) {
        coins.add(toCoins(line))
    }
}

```

```

        line = readFile.readLine()
    }
    return coins.toTypedArray()
}
/**
 * Function: writeStations()
 *
 * Description: Writes an array of stations to a file
 * @param fileName Name of the file to write
 * @param stations Array of Stations to be written
 * @return void
 * @see Stations
 */
fun writeStations(fileName: String, stations: Array<Stations>) {
    val writeFile = createWriter(fileName)
    for (i in stations) {
        writeFile.println("${i.price};${i.sold};${i.station}")
    }
    writeFile.close()
}
/**
 * Function: writeCoins()
 *
 * Description: Writes an array of coins to a file
 * @param fileName Name of the file to write
 * @param coins Array of Coins to be written
 * @return void
 * @see Coins
 */
fun writeCoins(fileName: String, coins: Array<Coins>) {
    val writeFile = createWriter(fileName)
    for (i in coins) {
        writeFile.println("${i.value};${i.amount}")
    }
    writeFile.close()
}
}

//=====
//
//
//=====
fun main() {
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} File Writer & File Reader")
}

```

```

print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
Time.sleep(1000)
print("")
Time.sleep(1000)
print("\n")
Time.sleep(1000)
println("=====")
while (true) {
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Choose a task: ")
    println("1 - Write predefined Arrays")
    println("2 - Read existing files")
    println("3 - Write your own Data")
    println("D - Quit")
    print(">${Miscellaneous.RESET} ")
    when (readln().uppercase()) {
        "1" -> {
            val a = arrayOf(
                FileAccess.Stations(150, 10, "Morangos com açúcar"),
                FileAccess.Stations(220, 5, "Matrix"),
                FileAccess.Stations(100, 3, "Dune"))
            val b = arrayOf(
                FileAccess.Coins(5, 50),
                FileAccess.Coins(10, 20),
                FileAccess.Coins(20, 30),
                FileAccess.Coins(50, 10),
                FileAccess.Coins(100, 15),
                FileAccess.Coins(200, 5))
            FileAccess.writeStations("testStations.txt", a)
            FileAccess.writeCoins("testCoins.txt", b)
            println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Data written successfully! C
        }
        "2" -> {
            val s = FileAccess.readStations("testStations.txt")
            val c = FileAccess.readCoins("testCoins.txt")
            s.forEach { println("${Miscellaneous.YELLOW}Station: ${it.station}, Price: ${it.p
            c.forEach { println("${Miscellaneous.YELLOW}Coin: ${it.value}, Amount: ${it.amoun
        }
        "3" -> {
            println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Manual data Insertion")
            val stations = mutableListOf<FileAccess.Stations>()
            println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Insert Stations (format: pr
            while (true) {
                print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD}Station: ")

```



```

        val input = readln()
        if (input.uppercase() == "Q") break
        try { stations.add(FileAccess.toStations(input)) } catch (e: Exception) { pri
    }
    val coins = mutableListOf<FileAccess.Coins>()
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Insert Coins (format: value
    while (true) {
        print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD}Coin: ")
        val input = readln()
        if (input.uppercase() == "Q") break
        try { coins.add(FileAccess.toCoins(input)) } catch (e: Exception) { println("
    }
    if (stations.isNotEmpty()) FileAccess.writeStations("testStations.txt", stations.
    if (coins.isNotEmpty()) FileAccess.writeCoins("testCoins.txt", coins.toTypedArray
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Data written successfully! C
    }
    "D" -> break
    else -> println("${Miscellaneous.RED}${Miscellaneous.BOLD} Invalid option, try again.
    }
}
}
}

```

B.8 KBD

```

import isel.leic.UsbPort
import isel.leic.utils.Time

```

```

//=====
//                                     Keyboard
//=====
object KBD {
    /**NONE refers to invalid key.
     *
     * none converts to a null character (\u0000), since 0 is a valid key code.**/
    const val NONE = 0;
    val none = NONE.toChar()
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see KeyReceiver.init
    */
}

```

```

    */
fun init() {
    KeyReceiver.init()
}
/**
 * Function: getKey()
 *
 * Description: Instantly decodes and gives corresponding char.
 * @param void
 * @return keyConvert. This returns the corresponding char of the key code.
 * @see KeyReceiver.serialReceiver
 */
fun getKey(): Char {
    val keyCode = KeyReceiver.serialReceiver()
    val keyConvert =
        when (keyCode) {
            0 -> '1'
            1 -> '4'
            2 -> '7'
            3 -> '*'
            4 -> '2'
            5 -> '5'
            6 -> '8'
            7 -> '0'
            8 -> '3'
            9 -> '6'
            10 -> '9'
            11 -> '#'
            12 -> 'A'
            13 -> 'B'
            14 -> 'C'
            15 -> 'D'
            else -> none
        }
    return keyConvert
}
/**
 * Function: waitKey()
 *
 * Description: Returns key if pressed before timeout,
 * otherwise returns none.
 * @param timeout Refers to duration, in milliseconds, of a key detection.
 * @return Key or none.

```

```

    * @see KBD.getKey
    * @see System.currentTimeMillis
    */
fun waitKey(timeout: Long): Char {
    val start = System.currentTimeMillis()
    while (System.currentTimeMillis() - start < timeout) {
        val key = getKey()
        if (key != none) {
            return key
        }
    }
    return none
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    KBD.init()
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} To test Ring buffer: Turn software off and on")
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Test different repeat intervals with Switch")
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("===== ${Miscellaneous.CYAN}${Miscellaneous.BOLD} Test")
    print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Keys pressed: ${Miscellaneous.GREEN} ")
    while(true) {
        print("${KBD.waitKey(100000)}, ")
    }
}

```

B.9 Key Receiver

```
import isel.leic.utils.Time
```

```

//=====
//                                     Key Receiver
//=====
object KeyReceiver {

```

```

/**
 * Function: init()
 *
 * Description: This functions inits the object
 * @param void
 * @return void
 * @see HAL.init
 * @see HAL.clrBits
 */
fun init() {
    HAL.init()
    HAL.clrBits(OUTPUTPORTS.TXclk.mask)
}

/**
 * Function: serialReceiver()
 *
 * Description: Builds the frame as data is received serially.
 * Detects protocol errors and isolates the KeyCode.
 * @param void
 * @return Key Code or -1
 * @see HAL.isBit
 * @see HAL.clrBits
 */
fun serialReceiver(): Int {
    if (HAL.isBit(INPUTPORTS.TXD.mask)) return -1
    var frame = 0
    for (i in 0 until 6) {
        HAL.setBits(OUTPUTPORTS.TXclk.mask)
        HAL.clrBits(OUTPUTPORTS.TXclk.mask)
        val TXD = if (HAL.isBit(INPUTPORTS.TXD.mask)) 1 else 0
        frame = frame or (TXD shl i)
    }
    HAL.setBits(OUTPUTPORTS.TXclk.mask)
    HAL.clrBits(OUTPUTPORTS.TXclk.mask)
    val start = frame and 0b000001
    val stop = (frame and 0b100000).shr(5)
    val key = frame and 0b011110
    if (start == 0) return -1
    if (stop == 1) return -1
    else return (key shr 1) and 0b1111
}
}

//=====

```

```
//
//=====
//
//                                TESTBENCH
//=====
fun main() {
    KeyReceiver.init()
    TUI.init()
    println("${Miscellaneous.RED}${Miscellaneous.BOLD} Make sure to not spam keys, ring buffer wi
    println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Insert sleep functions on serial receive
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====")
    while (true) {
        Time.sleep(2000)
        println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Press a key${Miscellaneous.RESET}")
        Time.sleep(5000)
        val keyCode = KeyReceiver.serialReceiver()
        if (keyCode == -1) {
            println("${Miscellaneous.RED}Code: -1 | None")
            TUI.write("Code: -1 | None", 0, 0, true, true)
        }
        else {
            println("${Miscellaneous.GREEN}Code: $keyCode | ${keyCode.toString(2).padStart(4, '0')}")
            TUI.write("Code: $keyCode | ${keyCode.toString(2).padStart(4, '0')}", 0, 0, true, tru
        }
    }
}
}
```

B.10 HAL

```
import isel.leic.UsbPort
import isel.leic.utils.Time
```

```
//=====
//
//                                HARDWARE ABSTRACT LAYER
//=====
object HAL {
    /**Output Port state (protected)**/
    private var usbport = 0
    /**
     * Function: init()
    */
}
```

```

*
* Description: This functions inits the object
* @param void
* @return void
* @see UsbPort
*/
fun init() {
    usbport = 0
    UsbPort.write(usbport)
}
/**
* Function: readBits()
*
* Description: This functions reads the value of the bits,
* filtered by mask.
* @param mask Filter out the bit/s of interest.
* @return UsbPort.read() and mask. This is the value written on Input ports.
* @see UsbPort
*/
fun readBits(mask: Int): Int {
    return UsbPort.read() and mask
}
/**
* Function: isBit()
*
* Description: Checks the logic value of a bit
* @param mask Filter out the bit/s of interest.
* @return (UsbPort.read() and mask != 0). This is the logic value of the bit filtered.
* @see UsbPort
*/
fun isBit(mask: Int): Boolean {
    return (UsbPort.read() and mask != 0)
}
/**
* Function: setBits()
*
* Description: Sets the filtered bits with logic value '1'
* @param mask Filter out the bit/s of interest.
* @return void
* @see UsbPort
*/
fun setBits(mask: Int) {
    usbport = usbport or mask
}

```

```

        UsbPort.write(usbport)
    }
    /**
     * Function: clrBits()
     *
     * Description: Sets the filtered bits with logic value '0'
     * @param mask Filter out the bit/s of interest.
     * @return void
     * @see UsbPort
     */
    fun clrBits(mask: Int) {
        usbport = usbport and mask.inv()
        UsbPort.write(usbport)
    }
    /**
     * Function: writeBits()
     *
     * Description: Sets the filtered bits with 'value'
     * @param mask Filter out the bit/s of interest.
     * @param value Value to be written on bits of interest
     * @return void
     * @see UsbPort
     */
    fun writeBits(mask: Int, value: Int) {
        usbport = (usbport and mask.inv()) or (value and mask)
        UsbPort.write(usbport)
    }
}

//=====
//                                     TESTBENCH
//=====
fun main() {
    HAL.init()
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Choose a task:${Miscellaneous.YELLOW}")
    println("0..7 - Toggle bit")
}

```

```
println("8 - Read Port")
println("9 - Set masked bits")
println("10 - Clear masked bits")
println("11 - Quit")
println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD}=====")
var quit = false
while (!quit) {
    val portState = UsbPort.read()
    print("\r${Miscellaneous.GREEN}${Miscellaneous.BOLD} Current Port State: 0b${portState.to")
    print("\r${Miscellaneous.GREEN}${Miscellaneous.BOLD}> ")
    val key = readln().toInt()
    when {
        key in 0..7 -> {
            val bitIndex = key
            val mask = 1 shl bitIndex
            if (HAL.isBit(mask)) HAL.clrBits(mask) else HAL.setBits(mask)
            println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Toggled bit $bitIndex")
        }
        key == 8 -> {
            print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}Mask to read (8 bits): ")
            val mask = readln().toIntOrNull(2) ?: 0
            println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Port read: ${HAL.readBi")
        }
        key == 9 -> {
            print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}Mask to set (8 bits): ")
            val mask = readln().toIntOrNull(2) ?: 0
            HAL.setBits(mask)
        }
        key == 10 -> {
            print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}Mask to clear (8 bits): ")
            val mask = readln().toIntOrNull(2) ?: 0
            HAL.clrBits(mask)
        }
        key == 11 -> quit = true
    }
    Time.sleep(100)
}
}
```

B.11 Serial Emitter

```
import isel.leic.utils.Time
```



```
//=====
//
//=====
object SerialEmitter {
    enum class Peripheral {LCD, TICKET}
    /**
     * Function: init()
     *
     * Description: This functions inits the object
     * @param void
     * @return void
     * @see HAL.init
     * @see HAL.setBits
     * @see HAL.clrBits
     */
    fun init() {
        HAL.init() //port a 0
        HAL.setBits(OUTPUTPORTS.SS_LCD.mask)
        HAL.setBits(OUTPUTPORTS.SS_TD.mask)
        HAL.clrBits(OUTPUTPORTS.SCLK.mask)
    }
    /**
     * Function: send()
     *
     * Description: This functions sends frames of 10 bits of data to a
     * specified peripheral
     * @param addr Peripheral of interest
     * @param data Data to be sent
     * @return void
     * @see OUTPUTPORTS
     * @see HAL.setBits
     * @see HAL.clrBits
     */
    fun send(addr: Peripheral, data: Int) {
        val SS = if (addr == Peripheral.LCD) OUTPUTPORTS.SS_LCD.mask else OUTPUTPORTS.SS_TD.mask
        HAL.clrBits(SS)
        for (i in 0 ..9) {
            HAL.clrBits(OUTPUTPORTS.SCLK.mask)
            val SDX = data shr i and 1
            if (SDX == 1) {
                HAL.setBits(OUTPUTPORTS.SDX.mask)
            } else HAL.clrBits(OUTPUTPORTS.SDX.mask)
            HAL.setBits(OUTPUTPORTS.SCLK.mask)
        }
    }
}
```

```

    }
    HAL.setBits(SS)
}
}
//=====
//                                TESTBENCH
//=====
fun main() {
    SerialEmitter.init()
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Write the data you want to be sent on Per
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Data is recommended to be written on 10
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====${Miscellaneous.CYAN}${Miscellaneous.BOLD}=====")
    while (true) {
        print("${Miscellaneous.GREEN}Data to LCD: ")
        val dataLCD = readln()
        val dataLCDbin = if (dataLCD.length == 10 && dataLCD.all { it == '0' || it == '1' }) dataLCD
        print("${Miscellaneous.GREEN}Data to TD: ")
        val dataTD = readln()
        val dataTDbin = if (dataTD.length == 10 && dataTD.all { it == '0' || it == '1' }) dataTD
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, dataLCDbin)
        SerialEmitter.send(SerialEmitter.Peripheral.TICKET, dataTDbin)
        print("${Miscellaneous.GREEN} Sent!\n")
        println("${Miscellaneous.GREEN}${Miscellaneous.BOLD}-----")
    }
}
}

```

B.12 Ticket Dispenser

```

import isel.leic.utils.Time
import kotlin.system.exitProcess

```

```

//=====
//                                TICKET DISPENSER
//=====
object TicketDispenser {
    /**
     * Function: init()
    */
}

```

```

*
* Description: This functions inits the object
* @param void
* @return void
* @see SerialEmitter.init
*/
fun init() {
    SerialEmitter.init()
}
/**
* Function: activatePrintingTicket()
*
* Description: Sends a command to print a ticket.
* @param roundTrip Trip type
* @param origin Origin station
* @param destination Destination station
* @return void
* @see SerialEmitter.send
* @see shl
*/
fun activatePrintingTicket(roundTrip: Boolean, origin: Int, destination: Int) {
    val RT = if (roundTrip) 1 else 0
    val data = RT or destination.shl(1) or origin.shl(5)
    val printData = RT or destination.shl(1) or origin.shl(5) or (1 shl 9)
    SerialEmitter.send(SerialEmitter.Peripheral.TICKET, data)
    SerialEmitter.send(SerialEmitter.Peripheral.TICKET, printData)
}
/**
* Function: lowerPrt()
*
* Description: Sends a command with Prt to low. This prevents the system
* from freezing when collecting a ticket.
* @param roundTrip Trip type
* @param origin Origin station
* @param destination Destination station
* @return void
* @see SerialEmitter.send
* @see shl
*/
fun lowerPrt(roundTrip: Boolean, origin: Int, destination: Int) {
    val RT = if (roundTrip) 1 else 0
    val data = RT or destination.shl(1) or origin.shl(5)
    SerialEmitter.send(SerialEmitter.Peripheral.TICKET, data)
}

```

```

    }
}

//=====
//                                     TESTBENCH
//=====

fun main() {
    TicketDispenser.init()
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Write the data you want to be sent")
    println("${Miscellaneous.YELLOW}${Miscellaneous.BOLD} Data is recommended to be written on bi
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("=====")
    while (true) {
        print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}RoundTrip:${Miscellaneous.GREEN} ")
        val rt = readln().toBoolean()
        print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}Origin:${Miscellaneous.GREEN} ")
        val origin = readln()
        print("${Miscellaneous.GREEN}${Miscellaneous.BOLD}Destination:${Miscellaneous.GREEN} ")
        val destination = readln()
        val originBin = if (origin.length == 4 && origin.all { it == '0' || it == '1' }) origin.t
        val destinationBin = if (destination.length == 4 && destination.all { it == '0' || it ==
        TicketDispenser.activatePrintingTicket(rt, originBin, destinationBin)
        println("${Miscellaneous.GREEN}${Miscellaneous.BOLD} Sending...")
        Time.sleep(3000)
        TicketDispenser.lowerPrt(false, 0, 0)
        print("${Miscellaneous.YELLOW}${Miscellaneous.BOLD}Continue [y/n]?${Miscellaneous.YELLOW}
        val next = readln()
        if (next.lowercase() != "y") exitProcess(0)
        println("-----")
    }
}

```

B.13 LCD

```
import isel.leic.utils.Time
```

```

//=====
//                                     LIQUID CRYSTAL DISPLAY

```

```
//=====
object LCD {
    /**Display Dimension**/
    const val LINES = 2
    const val COLS = 16
    /**
     * Protected Function: writeByteSerial()
     *
     * Description: Serially writes a command/data byte on LCD,
     * through a serial emitter
     * @param rs Register select - Selection between a command and a data
     * @param data data to be sent
     * @return void
     * @see SerialEmitter.send
     */
    private fun writeByteSerial(rs: Boolean, data: Int) {
        val rsBit = if (rs) 1 else 0
        val frame1 = rsBit or (data shl 1) or (1 shl 9)
        val frame2 = rsBit or (data shl 1)
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, frame1)
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, frame2)
    }
    /**
     * Protected Function: writeByte()
     *
     * Description: Writes a command/data byte on LCD.
     * @param rs Register Select - Selection between a command and a data
     * @param data data to be sent
     * @return void
     * @see writeByteSerial
     */
    private fun writeByte(rs: Boolean, data: Int) {
        writeByteSerial(rs, data)
    }
    /**
     * Protected Function: writeCMD()
     *
     * Description: Writes a command on LCD.
     * @param data Command to be sent (RS on high)
     * @return void
     * @see writeByte
     */
    private fun writeCMD(data: Int) {

```

```

        writeByte(false, data)
    }
    /**
     * Protected Function: writeDATA()
     *
     * Description: Writes data on LCD.
     * @param data Data to be sent (RS on low)
     * @return void
     * @see writeByte
     */
    private fun writeDATA(data: Int) {
        writeByte(true, data)
    }
    /**
     * Protected Function: createChar()
     *
     * Description: Creates & registers special characters on LCD memory
     * @param location Memory address for char to be stored
     * @param pattern Array of the custom char which represents the matrix of pixels
     * @return void
     * @see writeDATA
     * @see writeCMD
     */
    private fun createChar(location: Int, pattern: IntArray) {
        val address = (location and 0x07) shl 3
        writeCMD(0x40 or address)
        for (i in 0..7) {
            writeDATA(pattern[i])
            Time.sleep(2)
        }
        writeCMD(0x80)
    }
    /**
     * Function: init()
     *
     * Description: This functions inits the LCD for communication, display mode,
     * and loads special chars. Sends the command sequence needed for 8-bit communication.
     * @param void
     * @return void
     * @see writeCMD
     * @see Time.sleep
     * @see createChar
     */

```

```

fun init() {
    writeCMD(0b0011_0000)
    Time.sleep(20)
    writeCMD(0b0011_0000)
    Time.sleep(20)
    writeCMD(0b0011_0000)
    writeCMD(0b0011_1000)
    writeCMD(0b0000_1000)
    writeCMD(0b0000_0001)
    writeCMD(0b0000_0110)
    writeCMD(0b0000_1111)
    createChar(0, euro)
    createChar(1, upArrow)
    createChar(2, downArrow)
    createChar(3, fullBlock)
    createChar(4, emptyBlock)
    createChar(5, upTri)
    createChar(6, rightTri)
    createChar(7, leftTri)
}

/**
 * Function: write()
 *
 * Description: writes a Char in current cursor position
 * @param c Char to be displayed
 * @return void
 * @see writeDATA
 */
fun write(c: Char) {
    writeDATA(c.code)
}

/**
 * Function: write()
 *
 * Description: writes a String/Char sequence from current cursor position
 * @param text String to be displayed
 * @return void
 * @see writeDATA
 */
fun write(text: String) {
    for (i in text)
        writeDATA(i.code)
}

```

```

/**
 * Function: cursor()
 *
 * Description: Positions the cursor on specified line & column
 * @param line line to position
 * @param column column to position
 * @return void
 * @see writeCMD
 */
fun cursor(line: Int, column: Int) {
    writeCMD((line * 0x40 + column) or 0x80)
}

/**
 * Function: hideCursor()
 *
 * Description: Hides the cursor (misc)
 * @param void
 * @return void
 * @see writeCMD
 */
fun hideCursor() {
    LCD.writeCMD(0x0C)
}

/**
 * Function: showCursor()
 *
 * Description: Shows the cursor (misc)
 * @param void
 * @return void
 * @see writeCMD
 */
fun showCursor() {
    LCD.writeCMD(0x0F)
}

/**
 * Function: clear()
 *
 * Description: Sends command to clear the display and set the cursor
 * on coordinates (0,0)
 * @param void
 * @return void
 * @see writeCMD
 * @see cursor

```



```

        */
    fun clear() {
        writeCMD(0x01)
        cursor(0, 0)
    }
}

//=====
//
//                                     TESTBENCH
//=====

fun main() {
    LCD.init()
    println("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Write the text you want to see displayed")
    print("${Miscellaneous.CYAN}${Miscellaneous.BOLD} Initializing")
    Time.sleep(1000)
    print("")
    Time.sleep(1000)
    print("\n")
    Time.sleep(1000)
    println("===== ${Miscellaneous.CYAN}${Miscellaneous.BOLD} Write the text you want to see displayed")
    while(true) {
        print("${Miscellaneous.YELLOW}Write: ")
        val text = readln()
        LCD.clear()
        LCD.write(text)
    }
}

```

B.14 Ports

```

//=====
//
//                                     USBPORT INPUTS & OUTPUTS
//=====

/**
 * Class: OUTPUTPORTS()
 *
 * Description: This enumerated class stores the masks
 * of the outports
 * @param mask Corresponding bit
 */
enum class OUTPUTPORTS(val mask: Int) {
    SDX(0b00000001),

```

```

        SCLK(0b00000010),
        SS_LCD(0b00000100),
        SS_TD(0b00001000),
        ACCEPT(0b00010000),
        EJECT(0b00100000),
        COLLECT(0b01000000),
        TXclk(0b10000000),
    }
}
/**
 * Class: INPUTPORTS()
 *
 * Description: This enumerated class stores the masks
 * of the inports
 * @param mask Corresponding bit
 */
enum class INPUTPORTS(val mask: Int) {
    COIN_ID0(0b00000001),
    COIN_ID1(0b00000010),
    COIN_ID2(0b00000100),
    COIN(0b00001000),
    FN(0b00010000),
    M_OUT(0b01000000),
    TXD(0b10000000),
}

```

B.15 Others

```

import java.text.DecimalFormat
import isel.leic.utils.Time

//=====
//
//=====
/**Utilities**/
object Utils {
    private val euroFormat = DecimalFormat("0.00")
    fun centsToEuros(value: Int): String {
        return euroFormat.format(value / 100.0)
    }
    fun readDigitIdx(firstDigit: Char, maxSize: Int): Int {
        var value = firstDigit.digitToInt()
        while (true) {
            val next = KBD.waitKey(5000L)

```

```

        if (next !in '0'..'9') break
        value = value * 10 + next.digitToInt()
        if (value >= maxSize) value %= maxSize
    }
    return value.coerceIn(0, maxSize - 1)
}
}
/**Miscellaneous Features**/
object Miscellaneous {
    const val RESET = "\u001B[0m"
    const val GREEN = "\u001B[32m"
    const val YELLOW = "\u001B[33m"
    const val CYAN = "\u001B[36m"
    const val RED = "\u001B[31m"
    const val BOLD = "\u001B[1m"
    fun writeScrollingText(text: String, row: Int, scrollIndex: Int, width: Int = 16) {
        if (text.length <= width) {
            TUI.write(text.padEnd(width), row, 0, false, false)
        } else {
            val extendedText = "$text      $text"
            val offset = scrollIndex % (text.length + 6)
            val sliced = extendedText.substring(offset, offset + width)
            TUI.write(sliced, row, 0, false, false)
        }
    }
}
}

```

B.16 Custom Chars

```

//=====
//                                     CUSTOM CHARS
//=====
/**Pattern for euro**/
val euro = intArrayOf(
    0b00110,
    0b01001,
    0b11100,
    0b01000,
    0b11100,
    0b01001,
    0b00110,
    0b00000
)

```

```

/**Pattern for up arrow**/
val upArrow = intArrayOf(
    0b00000,
    0b00100,
    0b01110,
    0b10101,
    0b00100,
    0b00100,
    0b00100,
    0b00000
)

/**Pattern for down arrow**/
val downArrow = intArrayOf(
    0b00000,
    0b00100,
    0b00100,
    0b00100,
    0b10101,
    0b01110,
    0b00100,
    0b00000
)

/**Full Block**/
val fullBlock = intArrayOf(
    0b00000,
    0b00000,
    0b11111,
    0b11111,
    0b11111,
    0b11111,
    0b00000,
    0b00000
)

/**Empty Block**/
val emptyBlock = intArrayOf(
    0b00000,
    0b00000,
    0b11111,
    0b10001,
    0b10001,
    0b11111,
    0b00000,
    0b00000
)

```

```
)
/**Up Triangle**/
val upTri = intArrayOf(
    0b00100,
    0b01110,
    0b01110,
    0b11111,
    0b11111,
    0b11111,
    0b11111,
    0b00000
)
/**Right Triangle**/
val rightTri = intArrayOf(
    0b10000,
    0b11000,
    0b11100,
    0b11110,
    0b11100,
    0b11000,
    0b10000,
    0b00000
)
/**Left Triangle**/
val leftTri = intArrayOf(
    0b00001,
    0b00011,
    0b00111,
    0b01111,
    0b00111,
    0b00011,
    0b00001,
    0b00000
)
/**Mini Triangle**/
val miniTri = intArrayOf(
    0b00000,
    0b10000,
    0b11000,
    0b11100,
    0b11000,
    0b10000,
    0b00000,
```

0b00000

)

Entradas	pino
Rows[0]	W5
Rows[1]	AA14
Rows[2]	W12
Rows[3]	AB12
Osc	P11
CoinId[0]	C10
CoinId[1]	C11
CoinId[2]	D12
Coin	C12
M	A12
Tdelay[0]	B12
Tdelay[1]	A13
CT	B14
RESET	F15
Saídas	pino
Cols[0]	AB11
Cols[1]	AB10
Cols[2]	AA9
Cols[3]	AA8
Accept	A8
Eject	A9
Collect	A10
Prt	B11
D9	V5
D[0]	W8
D[1]	AA15
D[2]	W13
D[3]	AB13
D[4]	Y11
D[5]	W11
D[6]	AA10
D[7]	Y8
D[8]	Y7
HEX0[0]	C14
HEX0[1]	E15
HEX0[2]	C15
HEX0[3]	C16
HEX0[4]	E16
HEX0[5]	D17
HEX0[6]	C17
HEX0[7]	D15
HEX1[0]	C18
HEX1[1]	D18
HEX1[2]	E18
HEX1[3]	B16
HEX1[4]	A17
HEX1[5]	A18
HEX1[6]	B17
HEX1[7]	A16
HEX2[0]	B20
HEX2[1]	A20
HEX2[2]	B19
HEX2[3]	A21
HEX2[4]	B21
HEX2[5]	C22
HEX2[6]	B22
HEX2[7]	A19
HEX3[0]	F21
HEX3[1]	E22
HEX3[2]	E21
HEX3[3]	C19
HEX3[4]	C20
HEX3[5]	D19
HEX3[6]	E17
HEX3[7]	D22
HEX4[0]	F18
HEX4[1]	E20
HEX4[2]	E19
HEX4[3]	J18
HEX4[4]	H19
HEX4[5]	F19
HEX4[6]	F20
HEX4[7]	F17
HEX5[0]	J20
HEX5[1]	K20
HEX5[2]	L18
HEX5[3]	N18
HEX5[4]	M20
HEX5[5]	N19
HEX5[6]	N20
HEX5[7]	L19

Tabela 2: Atribuição de pinos para a FPGA [4]